

Does Observability Matter in Cloud-Native Systems? An Empirical Study on Real-Time Anomaly Detection

Ngoc Duc Anh Nguyen, Satish Kumar and Nawar Jawad

School of Built Environment, Engineering and Computing, Leeds
Beckett University, UK.

*Corresponding author(s). E-mail(s):

N.Nguyen3896@student.leedsbeckett.ac.uk; s.kumar@leedsbeckett.ac.uk;

Contributing authors: n.jawad@leedsbeckett.ac.uk;

Abstract

Cloud-native systems are observable but non-stationary: the telemetry that machine-learning detectors consume is continuously reshaped by deployments, autoscaling and shifting traffic, so a decision boundary that is accurate at training time silently decays in production. Existing studies of observability-driven anomaly detection almost universally evaluate a model that is trained once on a static snapshot, and therefore measure a *stationary ceiling* that may not survive contact with a drifting stream. This paper asks a different question: when the stream drifts, does the *learning paradigm*—static-batch, periodically-retrained, or continuously-online—matter more than the richness of the telemetry, and how do the two interact? We construct a production-representative testbed, TraceFlix, a Kubernetes microservice application instrumented end-to-end with OpenTelemetry, Prometheus, Loki and Tempo, and drive it through four ordered telemetry configurations (metrics-only through full Metrics–Events–Logs–Traces) and four operating regimes that inject realistic drift (latency regression, scale-out, and combined load). On a held-out baseline set, detection quality rises monotonically with telemetry richness (F1 **0.896** $\rightarrow$ **0.986**), with distributed traces the single most valuable signal and ensemble trees the strongest model family; traces also lift root-cause top-1 localisation from **0.77** to a perfect **1.0** on a deep nine-service mesh. But under drift the picture inverts: a frozen batch model collapses to F1 $\approx$ **0.36** *regardless* of how much telemetry it is given, scheduled retraining recovers most of the gap (up to $\sim$ **0.93**), and a lightweight online-adaptive detector dominates once the telemetry is rich enough—decisively at C3/C4 and in two-thirds of operating regimes—reaching F1 $\approx$ **0.98** under

full telemetry and even exceeding an all-regimes batch oracle, while under thin telemetry it runs within **0.007** F1 of periodic retraining. The online detector is additionally **9–20**× better on tail latency, $\sim$ **130–390**× smaller, and retains no historical data. We conclude that telemetry completeness and continual learning are complementary rather than substitutable: richer signals raise the attainable ceiling, but only adaptation realises it on a live, drifting system.

Keywords: Observability, Anomaly Detection, Concept Drift, Online Learning, Cloud-Native Systems, Machine Learning, Microservices

1 Introduction

1.1 The Problem and Its Significance

Cloud-native systems—collections of loosely coupled microservices deployed on container orchestrators such as Kubernetes—have become the dominant architecture for large-scale online services [1]. Their elasticity and rate of change are precisely what make them difficult to operate. A modern deployment may roll out new code many times per day [2], autoscale pods up and down in response to traffic, reschedule workloads across heterogeneous nodes, and degrade partially in ways that never fully crash any single component. The very properties that make these systems resilient also make their failures diffuse, intermittent and hard to attribute [3]. Traditional threshold-based monitoring, in which an operator hand-codes a static alarm for each metric, scales neither to the number of signals nor to the rate at which their normal ranges shift [4, 5].

The response of the field has been *observability*: the practice of instrumenting a system so richly that its internal state can be reconstructed from the telemetry it emits [6, 7]. The telemetry is conventionally organised into the MELT model—Metrics, Events, Logs and Traces—and a mature observability stack federates all four through vendor-neutral instrumentation such as OpenTelemetry and storage backends such as Prometheus, Loki and Tempo [7, 8]. Observability turns the operational problem into a data problem, and a data problem invites machine learning. A substantial literature now applies classical, deep and graph-based models to observability data, both to *detect* that something has gone wrong and to *localise* the responsible service [9–14].

Two assumptions run, mostly unexamined, through this literature. The first is that *more telemetry is better*: that adding logs, then traces, then events monotonically improves detection. The second, more consequential, assumption is that the world is *stationary*: that a model trained once on a representative snapshot will continue to perform once it is deployed. Almost every reported result is obtained by training a model on a fixed dataset and evaluating it on a held-out portion of the *same* distribution [9–11]; the few datasets in common use are themselves narrow, and even on them headline numbers prove fragile under stricter evaluation [15, 16]. This protocol measures what we will call the *stationary ceiling*—the best a detector can do when nothing changes. It is silent on what happens afterwards, when a deployment shifts a service’s latency profile, an autoscaler doubles its replica count, or a traffic surge

moves every metric’s baseline at once. In those moments the data-generating distribution drifts, and a decision boundary fitted to the old normal begins to mislabel the new normal as anomalous.

This paper takes that second assumption as its subject. We argue that for a non-stationary observability stream the dominant design variable is not how much telemetry a detector consumes but *whether and how it keeps learning*. We make this concrete by studying two orthogonal axes together. The first axis is the familiar one of *telemetry completeness*: four ordered configurations from metrics-only to full MELT. The second, which the existing literature rarely treats as a variable at all, is the *learning paradigm*: a static-batch model trained once and frozen; a periodically-retrained model refitted on a sliding buffer; and an online-adaptive model that updates incrementally on every window. Crossing these axes lets us ask not only “does observability matter?” but “*when* does it matter, and compared to what?”

A concrete vignette fixes the problem. Suppose a team trains an anomaly detector on a representative week of telemetry and ships it; it performs beautifully in validation. A fortnight later the platform team rolls out a caching change that raises a service’s median latency by a modest, entirely healthy margin, and—independently—an autoscaler doubles the service’s replicas during a marketing push. Nothing is wrong, but every latency and per-replica-utilisation feature the detector consumes has shifted. The frozen detector, whose notion of “normal” was fixed a fortnight ago, now scores this healthy traffic as anomalous and pages the on-call engineer through the night. The team’s options are unappealing: silence the detector (losing real coverage), hand-tune thresholds (the very toil observability was meant to remove), or retrain—which lags the next such change by however long the retraining cycle is. This is not a contrived edge case; it is the ordinary weekly life of a cloud-native service, and it is the precise scenario our RQ3 isolates and measures.

Three observations motivate the study. First, instrumentation is not free. Active tracing imposes runtime overhead, and the volume, velocity and variety of telemetry impose real storage and processing costs [6, 17–19]. A practitioner choosing how deeply to instrument needs evidence on the *marginal* detection value of each additional pillar, not a blanket assurance that more is better. Second, the field’s most recent systematic surveys converge on the same gap from two directions: Raeiszadeh et al. [14] find that existing detectors “systematically fail” on adaptability to dynamic topologies, while Hrusto et al. [20] find that the great majority of techniques are validated on data that does not represent the drift and heterogeneity of real operations. Adaptability and realistic non-stationarity are exactly the two things a static, single-snapshot evaluation cannot probe. Third, industrial experience reports describe the failure mode directly: at IBM Cloud scale, threshold and static models were found inadequate precisely because “normal” is a moving target [4].

Whether that answer is “more telemetry” or “keep the model current” is not an academic distinction: it decides where a practitioner should spend a finite instrumentation budget and how a detector should be engineered. The governing question of this article is therefore:

Does observability matter for real-time anomaly detection in cloud-native systems—and if so, is it the amount of telemetry that decides detection quality, or whether the detector keeps learning as the system changes?

1.2 Research Questions

We approach this question with a controlled study that fixes the detection pipeline and varies, orthogonally, the telemetry it is given and the way it learns, on a single production-representative testbed under controlled fault injection and controlled drift. Because production is non-stationary, the operative measurements are taken on a drifting future stream; a controlled, single-distribution held-out slice—the stationary ceiling that single-snapshot studies report—serves only as a clean reference against which the field’s prevailing intuitions can be checked. The most basic of those intuitions is that more telemetry is better, so the first question we ask is:

RQ1. *Under non-stationary conditions, does the completeness of observability data—metrics-only, +logs, +traces, full MELT—improve the accuracy of anomaly detection, and which pillar contributes the largest marginal gain?*

RQ1 establishes whether, and through which pillar, richer telemetry raises detection once the stream is non-stationary. Detection is only half of operations, however: an alert becomes actionable only when the responsible service is named, and because traces alone carry the cross-service causal structure that metrics and logs lack, we isolate their contribution to localisation:

RQ2. *Which observability pillar makes significant contribution for root-cause localisation, holding all other signals constant?*

Completeness and localisation describe what the telemetry can do; they leave open the question that the non-stationarity of production makes decisive—not *how much* telemetry a detector consumes, but *whether it keeps learning* as the system changes. A boundary fitted to yesterday’s normal silently decays as deployments, autoscaling and shifting load reshape the telemetry. This is the crux of the article:

RQ3. *When the telemetry stream is non-stationary, how do the static-batch, periodically-retrained and online-adaptive learning paradigms compare on detection quality and operational cost, and how does this comparison interact with telemetry completeness?*

RQ3 shows which learning paradigm production demands; that settles the operational question but leaves a modelling one, which we therefore take up last. Having established the online setting, we ask which model family the pipeline should use and what that choice implies for the streaming detector:

RQ4. *Which model family best exploits rich, multimodal observability features, and what does that choice imply for the model that must run in the online setting?*

RQ1 and RQ2 engage the established completeness and localisation intuitions of the observability literature, now under realistic non-stationarity, so that their results can

be read against prior expectations; RQ3 makes the learning paradigm the explicit variable, and RQ4 reconciles the offline model-selection evidence with the online detector that RQ3 shows to be necessary. Throughout, the *identical* pipeline and feature sets are re-used across the held-out reference and the drifting stream, so that any reversal of conclusions is attributable to drift and the learning paradigm rather than to a change of model, data, or task.

1.3 Contributions

To answer these questions we built *TraceFlix*, a Kubernetes microservice application instrumented end-to-end with OpenTelemetry, Prometheus, Loki and Tempo, and drove it through four ordered (nested) telemetry configurations and four operating regimes under controlled fault injection—a design that crosses *signal richness* with the *learning paradigm* while holding the model architecture, feature engineering, fault scenarios and random seed fixed. To our knowledge this is the first study to vary both axes together. The article makes the following contributions.

- **A two-axis experimental methodology** for observability-driven detection that isolates telemetry completeness (four nested configurations) and the learning paradigm (static, periodic, online) under controlled drift on a production-representative testbed (§3, §4).
- **Empirical findings that answer the research questions**, established over the full configuration $\times$ paradigm $\times$ regime grid:

To RQ1: under non-stationarity, telemetry completeness still improves detection: for a detector that keeps current, drifted-stream F1 rises *monotonically* from 0.81 to 0.98 across the configurations (mirroring $0.896 \rightarrow 0.986$ on a held-out reference), with distributed traces the single most valuable pillar—and decisively so under drift, where the trace increment (+0.146 F1) dwarfs that of logs or events.

To RQ2: adding traces lifts root-cause top-1 localisation from 0.77 to a perfect 1.0 on a deep nine-service mesh (where multi-hop latency leaves top- k unsaturated without traces), isolating tracing as the decisive localisation signal.

To RQ3: under drift the result *inverts*—a frozen batch model collapses to $F1 \approx 0.36$ regardless of telemetry, periodic retraining recovers much of the gap, and a lightweight online-adaptive detector dominates once telemetry is rich (C3/C4) and in 8 of 12 regime cells, reaching $F1 \approx 0.98$ and beating an all-regimes oracle at every configuration, while being 9–20 $\times$ better on tail latency and ~ 130 –390 $\times$ smaller; under thin telemetry it and periodic retraining are within 0.007 F1.

To RQ4: ensemble trees are the strongest family on multimodal tabular telemetry, which is also *why a lightweight normalised linear model—not a tree—is the right detector for the online setting* established by RQ3.

- **An operational-cost analysis** that reframes the choice of learning paradigm as a service-level rather than a purely accuracy decision (§5, §6).
- **A streaming system realisation and a local-LLM detector.** The online paradigm is deployed as an Apache Kafka event backbone [21] over a nine-microservice mesh with real per-service business logic, and the detector family is

extended with a local LLM (Qwen2.5-3B [22] via Ollama [23], optionally LoRA-tuned [24]) that reasons over *raw* signals on the same stream. We contribute its architecture and three accompanying flowcharts (§4.3); its served-model evaluation is reported as ongoing, and we do not claim an accuracy figure for it.

- **Practical guidance** on minimum-viable observability and on when continual learning is mandatory rather than optional (§6).

To support open science, the testbed configuration, pipeline code and result datasets underpinning every table are available for reproducibility.

The remainder of this article is organised as follows. Section 2 fixes the necessary background and develops a taxonomy of cloud-native anomaly detection that locates the study within the field. Section 3 formalises windowed detection, concept drift and the two-axis design. Section 4 describes the TraceFlix testbed, fault injection, the telemetry configurations and drift regimes, the competing learners, and the evaluation protocol. Section 5 reports the results, establishing the held-out baseline findings first and returning to the model-family question (RQ4) last, once the online setting is established. Section 6 distils the lessons learned and Section 7 examines the threats to validity. Section 8 situates the work within the related literature, and Section 9 concludes.

2 Background and a Taxonomy of Cloud-Native Anomaly Detection

Before formalising the study it is worth fixing vocabulary and, more usefully, drawing a map of the field the study inhabits. Cloud-native anomaly detection is not a single technique but a family of them, and the choices that distinguish one detector from another—what telemetry it consumes, what model it fits, how it learns as the system changes, and what it is asked to decide—are precisely the choices this paper treats as variables rather than as fixed background assumptions. This section establishes the concepts the rest of the article relies on (§2.1) and organises the field into a taxonomy of detection approaches (§2.2, Figure 1) and locates within it the region this study occupies. The detailed, study-by-study comparison against the most directly related work is deferred to §8, once our own results are in hand.

2.1 Concepts and vocabulary

Observability and the MELT model.

A system is *observable* when its internal state can be reconstructed from its external outputs [6]. In a microservice deployment those outputs are telemetry, conventionally organised into the MELT model: *metrics* (numeric time series such as latency percentiles or CPU utilisation), *events* (discrete lifecycle records such as a pod restart), *logs* (timestamped, semi-structured text), and *traces* (per-request trees of spans that record how a request propagates across services) [7, 8]. The pillars differ both in operational cost and in the information they carry: metrics are cheap and aggregate, logs are semantically rich but unstructured, traces alone encode the cross-service causal structure of a request—and are the most expensive pillar to collect and store—and

events mark coarse state transitions. That a richer pillar costs more is exactly what makes the marginal detection value of each one an empirical question rather than a foregone conclusion.

The detection task and its relatives.

A detector consumes a window of telemetry and decides whether the system is behaving normally. Three operational tasks are usefully separated. *Detection* asks whether something is wrong; *localisation*, or root-cause analysis, asks which service or signal is responsible [11]; and *prediction* asks whether a violation is imminent, so that remediation can precede impact [5]. The three are sequential in operations—one must detect before localising—and they make different demands on the telemetry. This article concerns detection and localisation.

Non-stationarity and concept drift.

A cloud-native telemetry stream is *non-stationary*. *Concept drift* denotes a change over time in the joint distribution $P(\mathbf{x}, y)$ of features $\mathbf{x}$ and labels y [25, 26]. In operational terms, a deployment alters the telemetry that healthy behaviour produces, an autoscaling event shifts resource-utilisation baselines, and a traffic surge moves latency and throughput distributions at once; a decision boundary fitted to last week’s normal then misclassifies this week’s. Whether, and how, a detector copes with drift is therefore not a peripheral implementation detail but a defining property—and it is one of the two axes this study manipulates.

2.2 A taxonomy of cloud-native anomaly detection

Figure 1 organises cloud-native anomaly detection as a taxonomy of detection *approaches*, from simple statistical rules to multimodal deep models. Two further choices cut *across* this taxonomy—how a detector *learns* as the system changes, and whether it is *evaluated* under drift—and it is those two, rather than any single technique branch, that this paper treats as its variables.

The approaches differ chiefly in the function they fit. *Statistical and threshold* detectors raise an alarm when a signal leaves a hand-set or forecast band; they are cheap and interpretable but brittle as baselines move [4, 5]. *Classical machine learning*—tree ensembles, isolation forests and one-class methods—remains a strong baseline on tabular telemetry [9]. *Deep learning* dominates the recent literature in three sub-families: *reconstruction* models (autoencoders, variational autoencoders) that flag poorly-reconstructed windows [10]; *sequential* models (recurrent and transformer architectures) that capture temporal structure [27]; and *graph* models that encode the service topology and its span timings [11, 28, 29]. *LLM-based* detectors read raw logs and heterogeneous telemetry directly [30, 31], and *multimodal MELT-fusion* detectors combine all four pillars into a joint representation—the end at which the strongest recent results cluster [11, 32]. Cross-cutting every branch is the *task*: detection (is anything wrong?), localisation or root-cause analysis (which service?), and prediction (is a violation imminent?).

Two choices cut across every branch of this taxonomy and are easy to overlook yet decisive. The first is the *learning paradigm*: whether the chosen model is trained once

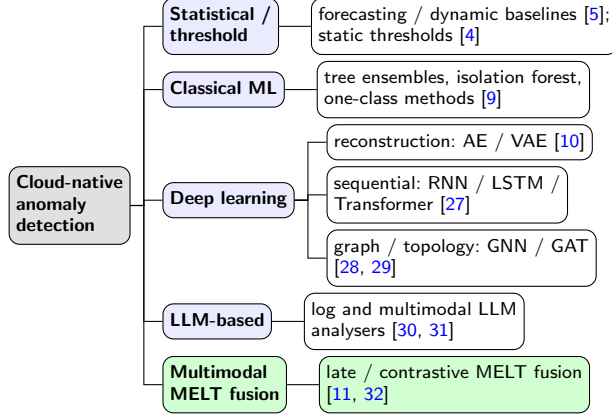


Fig. 1 A taxonomy of cloud-native anomaly-detection approaches—from statistical and threshold rules and classical machine learning, through the deep-learning families (reconstruction, sequential and graph models), to LLM-based and multimodal MELT-fusion detectors—with representative works. The strongest recent results cluster at the multimodal end (shaded). Cutting *across* this technique taxonomy are two choices the field rarely varies—how a detector *learns* as the system changes (static, periodic or online) and whether it is *evaluated* under drift—which §3 makes the explicit axes of this study.

and frozen (static), refitted on a sliding buffer at fixed intervals (periodic), or updated incrementally on every window (online) [33, 34]. It is orthogonal to the technique—any family above can in principle be deployed in any paradigm. The second is the *evaluation setting*: whether a detector is tested on a held-fixed (stationary, single-snapshot) slice of one distribution or on a moving (drift-aware) one [26], and whether its telemetry comes from a public benchmark or synthetic generator [35, 36], an operational production system [4, 20], or—the middle path this study takes—a production-representative testbed under controlled fault injection and controlled drift.

Read against this map, the field is unevenly worked. The technique branches are crowded and fast-moving: new modalities, architectures and tasks appear constantly, and the literature continues to advance each. The two cross-cutting choices, by contrast, are nearly collapsed. The overwhelming majority of cloud-native detectors are trained once and evaluated on a stationary slice of a single distribution [9–11], even though adaptability to a moving distribution is a named—and, by recent accounting, systematically unmet—requirement [14]. This paper therefore holds the detection technique fixed across its main comparisons (a deliberate control) and instead varies the two under-explored choices together—sweeping telemetry completeness and the learning paradigm under controlled drift, while also asking, last, which technique family best exploits the richest telemetry. It is that crossing—telemetry completeness with the learning paradigm—not progress within any single technique branch, that the study contributes, and the formal apparatus for it is the subject of the next section.

3 Problem Formulation

We now make the task and the two design axes precise: the *telemetry-completeness* axis and the *learning-paradigm* axis identified in the taxonomy of Figure 1, whose crossing was singled out there as the under-explored region this study occupies.

3.1 Windowed detection

Let the system emit a continuous multimodal telemetry stream. We discretise time into fixed-length, non-overlapping windows $t = 1, 2, \dots$. For each window a configuration-dependent feature-extraction function ϕ_c maps the raw telemetry in that window to a feature vector

$$\mathbf{x}_t^{(c)} = \phi_c(\text{telemetry}_t) \in \mathbb{R}^{d_c}, \quad (1)$$

where $c \in \{C1, C2, C3, C4\}$ indexes the telemetry configuration and d_c is the configuration’s feature dimensionality (Table 1). Each window carries a ground-truth binary label $y_t \in \{0, 1\}$ (1 = anomalous), derived from the fault-injection schedule (§4.4). A detector is a function $f_\theta : \mathbb{R}^{d_c} \rightarrow \{0, 1\}$ with parameters θ ; its prediction is $\hat{y}_t = f_\theta(\mathbf{x}_t^{(c)})$. Detection quality is scored by comparing $\{\hat{y}_t\}$ with $\{y_t\}$ using the metrics of §4.8.

Holding f , its hyper-parameter search, its preprocessing and its random seed fixed while varying only c isolates *signal availability* as the independent variable; this is the mechanism behind RQ1–RQ2. The configurations are *nested*, each a superset of the previous, so the marginal contribution of each added pillar is the incremental change between adjacent configurations.

Detection quality is quantified from the confusion counts over a window set: true positives TP (anomalous windows correctly flagged), false positives FP (normal windows flagged), true negatives TN and false negatives FN (missed anomalies). Precision, recall and their harmonic mean are

$$\text{Prec} = \frac{TP}{TP + FP}, \quad \text{Rec} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2 \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}. \quad (2)$$

F1 is the headline metric because the window labels are heavily imbalanced toward the normal class; under imbalance, accuracy $(TP+TN)/(TP+TN+FP+FN)$ is dominated by TN and a trivial always-normal predictor scores highly while detecting nothing. AUC-ROC complements F1 by summarising the precision–recall trade-off across all decision thresholds, characterising ranking quality independently of the operating point.

The decomposition into precision and recall is not merely descriptive; it is diagnostic of the failure mode we study. A detector whose decision boundary has become *stale* after drift does not fail symmetrically. Because the post-drift “normal” has moved into the region the boundary assigns to the anomalous class, the detector keeps firing on healthy traffic—recall on true anomalies may even stay high, but precision collapses as FP grows. The stale-normal failure is therefore visible as a *precision* collapse with preserved recall, and F1, by penalising the imbalance between the two, captures it where accuracy would not (§5.3.3).

3.2 Drift and the temporal axis

Let $P_t(\mathbf{x}, y)$ denote the joint distribution generating window t . We say the stream exhibits *concept drift* if $P_t \neq P_{t'}$ for some $t \neq t'$. The streaming-learning literature distinguishes two forms of drift [25], both of which our regimes induce. *Virtual drift* is a change in the feature distribution $P_t(\mathbf{x})$ with the conditional $P_t(y | \mathbf{x})$ unchanged—for example, a scale-out event shifts the resource-utilisation features of healthy pods without changing what “anomalous” means. *Real (concept) drift* is a change in $P_t(y | \mathbf{x})$ itself—for example, a latency level that was anomalous under the baseline becomes the new normal after a deployment regresses the service. Real drift is the more damaging for a static detector, because it directly invalidates the learned decision boundary rather than merely the input scaling; our regimes deliberately combine both so that neither a pure re-normalisation nor a pure re-fitting strategy would suffice alone. Drift is further characterised by its temporal profile—*abrupt* (a step change at a regime boundary) versus *gradual*; our regime transitions are abrupt, which is the harder case for periodic retraining because the lag between drift onset and the next scheduled refit is maximally exposed.

We operationalise drift as a sequence of *regimes* $R_0, R_1, \dots$, where within a regime the distribution is approximately stationary and between regimes an operational event (latency regression, scale-out, combined load; §4.6) shifts it. Regime R_0 is the baseline on which the offline models may train; the future stream R_1, R_2, R_3 is drifted relative to R_0 .

The temporal axis is the *learning policy* π that maps the observed prefix of the stream to the parameters used at each window:

- **Static** (π_{stat}): fit θ once on R_0 ; $\theta_t = \theta_0$ for all t .
- **Periodic** (π_{per}): refit θ every τ windows on the most recent B labelled windows; θ_t is piecewise-constant, updated at $t = \tau, 2\tau, \dots$.
- **Online** (π_{on}): update θ on every window via *prequential* (test-then-train) processing—predict $\hat{y}_t$ from θ_{t-1} , then update to θ_t using $(\mathbf{x}_t, y_t)$.

A reference *oracle* (π_{full}) is fit once on data drawn from *all* regimes; it is not realisable online (it sees the future) and serves only as an upper bound on what a single static boundary could achieve if drift were known in advance.

The online policy is evaluated *prequentially* (predictive-sequential): at each window the current model first predicts and is scored, then is updated with the now-revealed label. This protocol, standard in stream learning, has two properties we rely on. It uses every window for both testing and training without a held-out split, which is appropriate when data arrive as an unbounded stream; and, because each prediction is made strictly before the corresponding label is incorporated, it never leaks future information into a current prediction, so the reported online F1 is an honest estimate of what the deployed detector would have achieved in real time. The offline policies are scored on the same future windows for comparability: π_{stat} predicts all future windows from its R_0 -fit boundary, and π_{per} from whichever boundary was most recently refitted before each window.

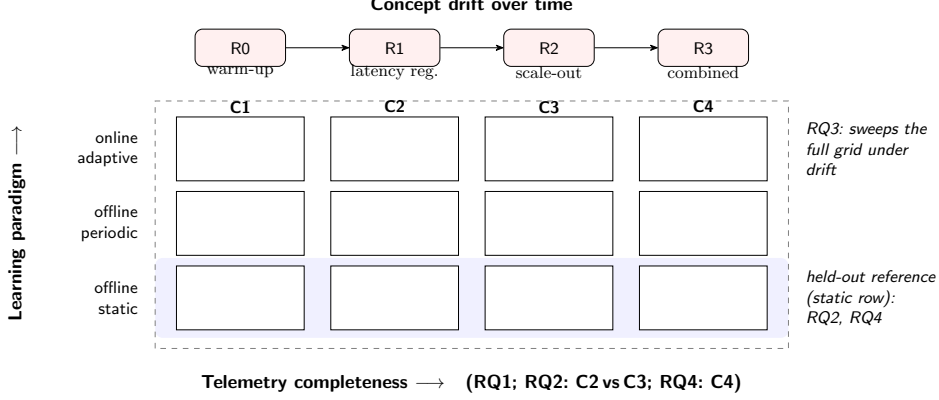


Fig. 2 The two-axis experimental design. The identical pipeline is evaluated over a grid of *telemetry completeness* (C1–C4, horizontal; each a superset of the previous) and *learning paradigm* (static / periodic / online, vertical). RQ1 reads detection along the completeness axis (on the drifted stream, with the static row as a held-out reference) and RQ2 reads localisation on that reference; RQ3 sweeps the whole grid on the drifted stream as the regimes $R_0 \rightarrow R_3$ shift the distribution.

3.3 The two-axis design

The study crosses the two axes, as Figure 2 depicts. The *completeness* axis sweeps $c \in \{C1-C4\}$: RQ1 reads it for detection—on the drifted future stream for the deployable continual-learning detector, with a held-out slice as a stationary reference—and RQ2 reads it for localisation on that held-out slice (the C2-vs-C3 contrast), while RQ4 fixes $c = C4$ and varies the model family. The *learning-paradigm* axis sweeps $\pi \in \{\pi_{\text{stat}}, \pi_{\text{per}}, \pi_{\text{on}}\}$ on the drifted future stream: RQ3 crosses it with c , with π_{full} as the oracle ceiling.

Because the feature extractor ϕ_c , the fault schedule, the load profile and the seed are identical across all cells, any difference in outcome is attributable to the cell’s coordinates— (c, π) —and not to a confound. This is the property that licenses the central claim: that the *collapse* of the static model under drift—and, conversely, the fact that the completeness benefit pays off only for a detector that keeps learning (RQ1, RQ3)—is caused by drift interacting with the learning policy, not by an incidental change of data or model.

4 Methodology

4.1 Research design and philosophy

The study adopts a positivist, deductive stance realised as a controlled, repeated-measures experiment, consistent with empirical software engineering. The relationships of interest are causal and comparative (“does X affect detection?”), so the design isolates each independent variable while holding confounds constant. Two factors are manipulated—telemetry completeness c and learning policy π —and the dependent variables are the detection-quality and operational-cost metrics of §4.8. The control

variables (model architecture, feature preprocessing, fault scenarios, load profile, random seed) are fixed across cells so that identical episodes are observed under differing configurations and policies. This within-subjects design is deliberate: because every cell sees the same underlying faults and the same drift schedule, differences cannot be ascribed to sampling variation between scenario sets.

4.2 The TraceFlix testbed

The platform is *TraceFlix*, a Kubernetes microservice application representing a realistic cloud-native system. The mesh comprises nine instrumented services with a genuine multi-hop dependency structure: a **gateway-service** entrypoint fans out to a **movie-service** subtree (**movie** issues N sequential calls to **actor-service** and a call to **review-service**) and to **user**, **search** and **recommendation** services that converge on a shared **catalog-service** leaf—a call graph of depth four with fan-out and a fan-in. A request therefore traverses a multi-hop caller/callee chain, and a fault deep in the graph induces secondary latency in *every* ancestor on its path. This exercises cross-service signal propagation for the localisation task on a graph large enough that top- k attribution is non-trivial (§5.2), yet controlled enough that the ground-truth origin is never in doubt.

Instrumentation is end-to-end and vendor-neutral, matching what a production OpenTelemetry deployment emits, which maximises the external validity of the telemetry schema [7, 20]:

- **Prometheus** — metrics: CPU, memory, network, request latency, error rates and custom application counters.
- **Loki** — structured application logs, container logs and Kubernetes event logs.
- **Tempo** — OpenTelemetry-instrumented distributed traces of service-to-service calls.
- **Kubernetes events** — OOMKilled and CrashLoopBackOff signals, the “E” of MELT.
- **Apache Kafka** [21] — an event backbone that carries per-window telemetry from TraceFlix to the detectors as a stream; it is the deployment realisation of the online pipeline, and the substrate the local-LLM detector also consumes (§4.3).

Figure 3 shows the end-to-end testbed, organised as the canonical observability stack [6]: an instrumented application emits telemetry, a vendor-neutral collector normalises it, per-pillar backends store it, and the detection pipeline consumes it. The four MELT pillars map directly onto the four telemetry configurations of §4.5.

4.3 Streaming realisation and the local-LLM detector

The online paradigm of §4.7 is not only an algorithm but a deployment: a detector that learns per window must be *fed* per window. We therefore realise the testbed as a streaming system and, on the same stream, add a qualitatively different detector—a local large language model that reasons over the *raw* telemetry rather than engineered features. Three flowcharts make the realisation concrete: the service call graph that

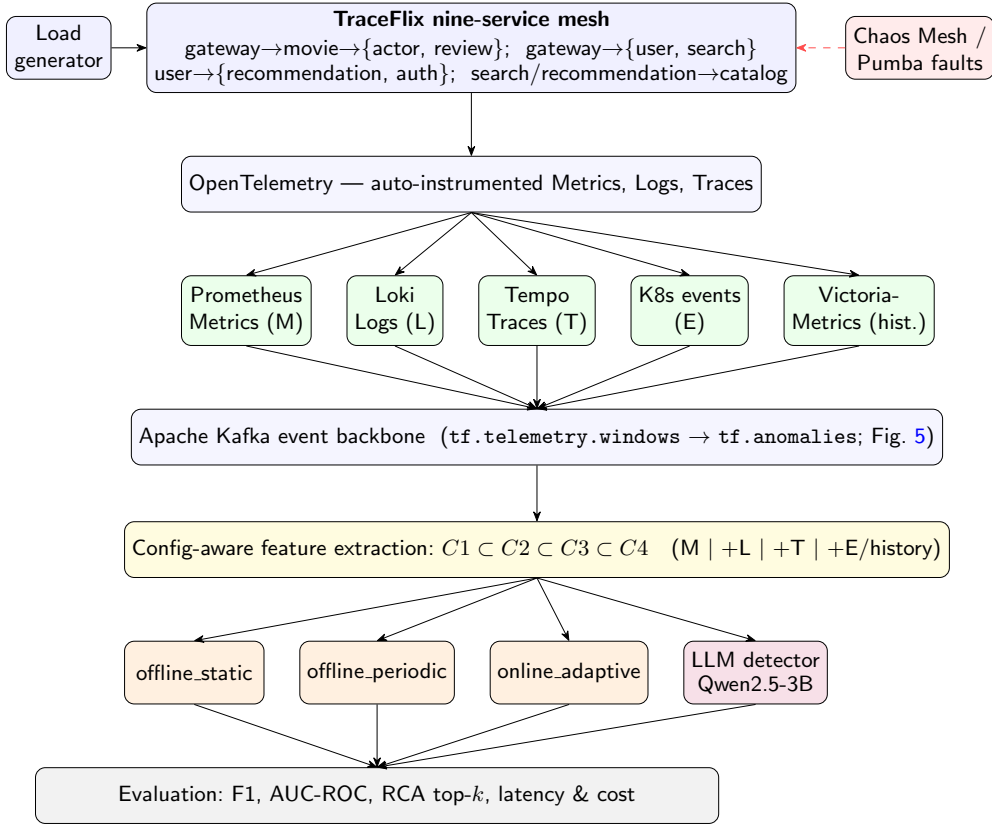


Fig. 3 The TraceFlix testbed and detection pipeline. A load generator drives the instrumented nine-service mesh (full call graph in Fig. 4); Chaos Mesh (or Pumba, on the Compose deployment) injects timestamped faults. OpenTelemetry collects MELT telemetry into per-pillar backends, whose signals flow over an *Apache Kafka* event backbone (Fig. 5) to a configuration-aware feature extractor ($C1 \subset C2 \subset C3 \subset C4$) feeding the three learning policies, a local-LLM detector (Qwen2.5-3B; Fig. 6), and the evaluation harness.

generates the telemetry (Fig. 4), the event backbone that transports it (Fig. 5), and the LLM detector that consumes it (Fig. 6).

The service mesh (Fig. 4).

The application is a nine-service mesh with a genuine, depth-four call graph and real per-service business logic, not a uniform stub. A **gateway** entrypoint aggregates a home page by fanning out to **movie** (which calls **actor** and **review**), **user** (which enriches a profile with **auth** and **recommendation**) and **search**; **recommendation** and **search** both read the shared **catalog** leaf. This depth is what makes localisation non-trivial: because a fault at a leaf propagates a secondary latency symptom up *every* ancestor on its path, several services are implicated at once, and only the originating-error-span feature disambiguates them (§5.2)—a structure a flat three-service chain cannot exhibit.

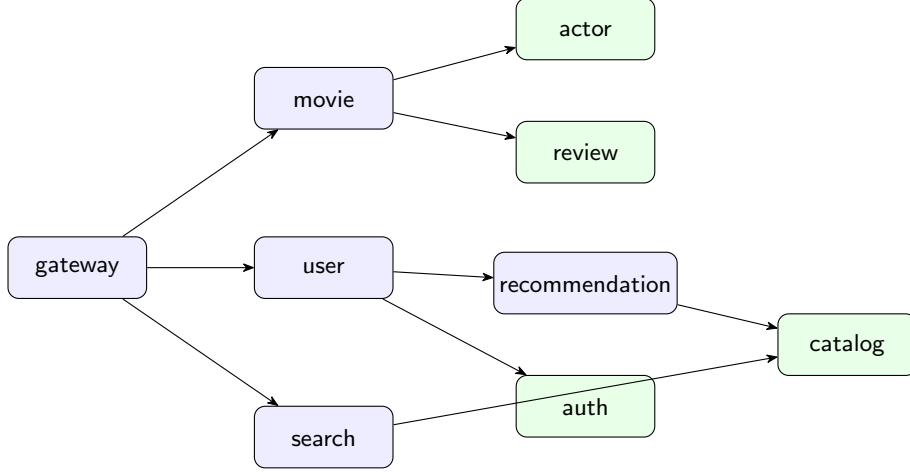


Fig. 4 The nine-service call graph (depth four). `gateway` is the entry; blue nodes are internal callers, green nodes are leaves. `catalog` is a shared *fan-in* reached by both `search` and `recommendation`. Data owners (`catalog`, `auth`, `user`) persist with Spring Data JPA; the rest orchestrate downstream calls. A fault at a leaf induces secondary latency in every ancestor on its path—the propagation RQ2 exploits.

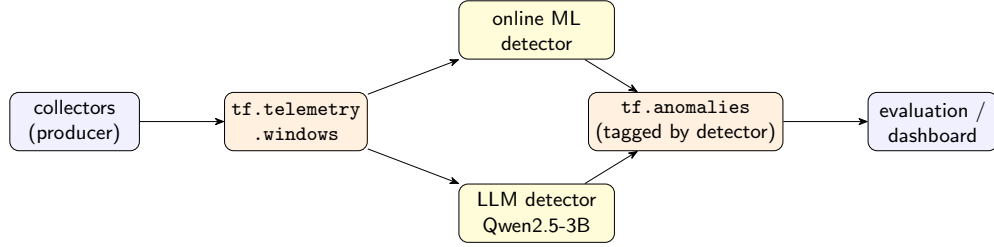


Fig. 5 The Kafka streaming dataflow. Per-window telemetry is produced to `tf.telemetry.windows`; the online ML detector and the local-LLM detector consume it in independent consumer groups and emit binary verdicts to `tf.anomalies`, tagged by detector for head-to-head comparison.

The Kafka event backbone (Fig. 5).

Telemetry is transported as an event stream on Apache Kafka [21]. A producer publishes one record per window to the `tf.telemetry.windows` topic; the online ML detector and the LLM detector consume it *independently*, each in its own consumer group, and publish a binary verdict to `tf.anomalies` tagged by detector, so the two are directly comparable on the same windows. Decoupling collection from detection from reasoning lets each stage scale and fail independently, and makes the per-window prequential protocol (test-then-train) a property of the transport rather than of a single process. For reproducibility the RQ3 results of §5.3 are computed on the *deterministic in-process* prequential harness; the Kafka backbone is the deployment realisation of the identical per-window protocol, and any broker-induced latency or loss it introduces is a deployment concern (§7), not a property of the reported numbers.

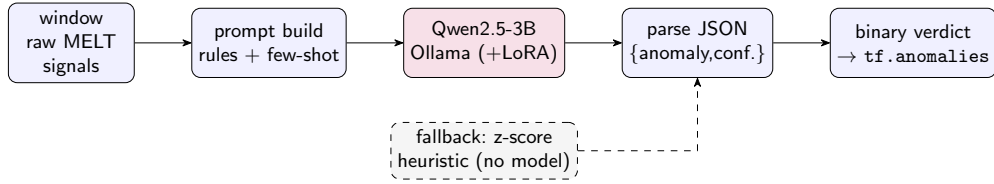


Fig. 6 The local-LLM detector pipeline. A window’s raw signals are formatted into a prompt (fault-signature rules + few-shot exemplars); Qwen2.5-3B, served by Ollama and optionally LoRA-tuned, returns a strict JSON binary verdict that is parsed to the same 0/1 output as the other detectors. When no model is served the pipeline falls back to a clearly-marked z-score heuristic (dashed), never silently substituted for the model.

The local-LLM detector (Fig. 6).

The second consumer is a local large language model, Qwen2.5-3B [22], served on-host by Ollama [23]. Rather than the engineered feature vector, it receives each window’s *raw* MELT signals, formatted into a compact prompt comprising a rule sheet of the six fault signatures and a few in-context exemplars sampled from the training split; it returns a strict JSON binary verdict `{anomaly, confidence}` that is parsed into the same 0/1 output as every other detector. An optional LoRA adapter [24] specialises the base model to this mesh’s signatures; the lift of tuning over the base prompt is the quantity of interest. When no model is served, the detector emits a *clearly-marked* z-score heuristic so that a fallback is never mistaken for a model run. This design connects the LLM-based detection thread of the literature [30, 31] to the streaming, drift-aware setting of this paper. **Evaluation status.** The detector is implemented and integrated into the prequential protocol, but a full quantitative evaluation with the *served* model requires the GPU deployment and is reported as ongoing work; we therefore present its architecture rather than a headline accuracy claim, and we are explicit that the heuristic fallback is not a proxy for the model (§7). Its intended role is complementary rather than competitive: §5.3 shows the per-window streaming detector must be *cheap and bounded*—properties a 3B-parameter model does not have—so the LLM is best positioned as an on-demand reasoning and triage layer over already-flagged windows, not as the primary per-window detector.

4.4 Fault injection and ground-truth labelling

Ground-truth labels are generated by controlled fault injection following the timestamped-injection methodology of Pedrosa et al. [37]. Faults are applied to running pods with precise start/stop timestamps, which become the labels against which detector output is scored. The fault taxonomy spans resource-exhaustion, performance-degradation and availability modes: memory leaks, CPU saturation, latency spikes, error bursts, OOMKilled events, CrashLoopBackOff, connection-pool exhaustion and network partitions. To produce genuine Kubernetes events for the “E” signal used by C4, container resource limits are patched so that memory-leak faults escalate to real OOMKills rather than simulated ones. A load generator drives continuous, realistic traffic so that each fault rides on a live request stream rather than an idle system; each *episode* selects a service as the fault origin, applies the fault, and

Table 1 The four telemetry configurations (completeness axis). Each is a strict superset of the previous, enabling marginal-contribution analysis.

Config	Signals	Features	Represents
C1	Metrics only	10	Basic infrastructure monitoring
C2	Metrics + Logs	14	Intermediate observability
C3	+ Traces	18	Full three-pillar observability
C4	Full MELT (+ Events + history)	23	Advanced observability

lets dependent services exhibit secondary effects (e.g. induced latency), reproducing realistic fault propagation for RCA.

4.5 Telemetry configurations (the completeness axis)

The completeness axis is operationalised as four ordered, nested configurations (Table 1). Feature engineering is *configuration-aware*: only the feature families permitted by the active configuration are assembled, so an identical telemetry window yields different feature vectors per configuration. This is the mechanism that isolates signal availability as the independent variable.

Feature engineering operates on the telemetry falling within each fixed-length window and aggregates it into a flat vector, so that all model families consume a common tabular representation. The feature families added by each pillar are summarised in Table 2 (the full per-feature listing is given in Appendix A). The metrics pillar (C1) contributes per-window aggregates of resource and request signals—CPU and memory utilisation, request and error rates, and latency percentiles—which are the signals available to any team with basic monitoring. Logs (C2) add volume- and severity-based features (error- and warning-log counts and a log-template diversity measure) that disambiguate anomalies whose metric signature is equivocal. Traces (C3) add span-level features—span rate, mean span duration, error-span rate—and, critically for localisation, the *originating-error-span* feature that identifies the service at which an error tree begins rather than the services that merely propagate its latency. Events and history (C4) add Kubernetes lifecycle-event counts (OOMKilled, CrashLoopBackOff, restarts) and rolling-baseline deviation features that encode how far the current window departs from a longer historical norm.

Two properties of this design matter for the validity of the completeness comparison. First, the feature families are *additive and disjoint*: enabling a pillar adds its features without altering those of lower pillars, so an identical telemetry window produces feature vectors that are strict prefixes of one another as c increases. Second, all features are window-local aggregates, deliberately avoiding high-cardinality raw identifiers (§8.2); this keeps the representation production-realistic and ensures the online model’s per-window update cost is bounded by a small, fixed dimensionality rather than growing with traffic.

Table 2 Feature families added by each pillar. Counts are cumulative across the nested configurations (cf. Appendix A).

Pillar	Added at	+Feat.	Representative features
Metrics	C1	10	CPU/mem util., req. rate, error rate, p50/p95/p99 latency, net I/O, restarts
Logs	C2	4	log volume, error-log count, warn-log count, template diversity
Traces	C3	4	span rate, mean span duration, error-span rate, originating-error-span
Events+history	C4	5	OOMKilled, CrashLoopBackOff, restart events, rolling-baseline deviations

Table 3 Learners instantiating the temporal axis.

Learner	Family	Learning policy
<code>offline_static</code>	Random Forest	Train once on R_0 , then frozen
<code>offline_periodic</code>	Random Forest	Refit every 500 windows on a 2,880-window buffer
<code>online_adaptive</code>	SGD + adaptive norm. + bandit	Prequential <code>partial_fit</code> , one window at a time
<code>offline_full</code>	Random Forest	Oracle: trained on all regimes (reference ceiling)

4.6 Drift regimes (the temporal axis)

Non-stationarity is induced as a controlled sequence of operating regimes, each representing a common operational event that shifts the telemetry distribution:

R_0 — *Baseline*. The nominal operating point; the offline models train here.

R_1 — *Latency regression*. A deployment-style shift that raises baseline service latency, moving the “normal” latency distribution.

R_2 — *Scale-out*. An autoscaling event that changes replica counts and therefore per-pod resource-utilisation baselines.

R_3 — *Combined load*. A traffic surge superimposed on the previous shifts—the hardest, most-drifted regime, moving multiple distributions at once.

The stream is processed prequentially over 320 episodes and 34,560 windows (nine services per episode-window), of which 8,640 constitute the R_0 warm-up and 25,920 the drifted future (R_1 – R_3) on which RQ3 is scored.

4.7 Learners and the online model

Four learners instantiate the temporal axis (Table 3). The offline learners use a Random Forest, chosen because the model-family comparison (RQ4, §5.4, which we report last) identifies ensemble trees as the strongest family on this multimodal tabular telemetry. The online learner is a streaming model built for prequential operation.

Algorithm 1 Online-adaptive prequential detection

```
1: Input: telemetry stream; candidate set  $\mathcal{H} = \{(\eta_0, \alpha)\}$ 
2: Initialise EW normaliser  $\mathcal{N}$ ; models  $\{m_h\}_{h \in \mathcal{H}}$ ; champion  $h^* \leftarrow \text{default}$ 
3: for each window  $t = 1, 2, \dots$  do
4:    $\mathbf{x}_t \leftarrow \phi_c(\text{telemetry}_t)$  ▷ configuration-aware features
5:    $\tilde{\mathbf{x}}_t \leftarrow \mathcal{N}.\text{TRANSFORM}(\mathbf{x}_t)$  ▷ rescale by running mean/var
6:    $\hat{y}_t \leftarrow m_{h^*}.\text{PREDICT}(\tilde{\mathbf{x}}_t)$  ▷ test before train
7:   emit  $\hat{y}_t$ 
8:   observe label  $y_t$ 
9:    $\mathcal{N}.\text{UPDATE}(\mathbf{x}_t)$ 
10:  for  $h \in \mathcal{H}$  do ▷ update all candidates
11:     $m_h.\text{PARTIAL\_FIT}(\tilde{\mathbf{x}}_t, y_t)$ 
12:    update rolling prequential F1 of  $m_h$ 
13:  end for
14:  if  $\arg \max_h \text{F1}(m_h) \neq h^*$  then ▷ drift-triggered re-selection
15:     $h^* \leftarrow \arg \max_h \text{F1}(m_h)$ ; record adapt event
16:  end if
17: end for
```

The online detector combines three mechanisms designed to track drift without retaining history (Algorithm 1). First, an incrementally-fitted linear classifier (stochastic gradient descent, logistic loss) updates on every window. Second, an exponentially-weighted (EW) running normaliser rescales each feature using running estimates of its mean and variance, so that a shift in a feature’s baseline—the signature of drift—is absorbed by the normaliser rather than corrupting the decision boundary. Third, a *champion bandit* maintains a small pool of candidate learning-rate/regularisation configurations and re-selects the current champion by recent prequential F1 whenever a candidate overtakes the incumbent, providing drift-triggered adaptation of the update dynamics themselves.

The three mechanisms have the following precise form. The classifier maintains a weight vector $\mathbf{w}$ updated by stochastic gradient descent on the logistic loss with ℓ_2 regularisation strength α and an inverse-scaling learning rate derived from η_0 ; the per-window update touches only the d_c active weights and is therefore $O(d_c)$ in both time and memory, with no dependence on stream length. The exponentially-weighted normaliser maintains, for each feature j , running estimates updated at every window with decay λ ,

$$\mu_{t,j} = (1 - \lambda) \mu_{t-1,j} + \lambda x_{t,j}, \quad (3)$$

$$s_{t,j}^2 = (1 - \lambda) s_{t-1,j}^2 + \lambda (x_{t,j} - \mu_{t,j})^2, \quad (4)$$

and rescales the input as $\tilde{x}_{t,j} = (x_{t,j} - \mu_{t,j}) / \sqrt{s_{t,j}^2 + \varepsilon}$. Because μ and s^2 track the moving feature distribution, a virtual-drift shift in a feature’s baseline (§3.2) is absorbed into the normaliser rather than presented to the classifier as a spurious anomaly—this

is the mechanism that protects against the stale-normal precision collapse. The champion bandit maintains the candidate pool $\mathcal{H} = \{(\eta_0, \alpha) : \eta_0 \in \{0.01, 0.05, 0.1\}, \alpha \in \{10^{-4}, 10^{-3}\}\}$, scores each candidate by its prequential F1 over a trailing window, and re-selects the champion h^* only when a challenger exceeds the incumbent by a margin δ , a hysteresis that prevents thrashing between near-tied candidates. Re-selection events are logged as “adapt events” and reported in §5.3.5; their frequency is itself an indicator of how hard the drift is to track at a given telemetry richness.

For RQ4 the model-family comparison additionally evaluates, on the richest configuration C4, Random Forest, two gradient-boosting implementations, an LSTM temporal model, and an attention-style late-fusion model drawing on the building-blocks strategy of HolisticRCA [11]. For RQ2 the localisation component ranks candidate fault entities, comparing a metrics-and-logs localiser (C2 features) against one that additionally consumes the trace-derived originating-error-span signal (C3 features), thereby isolating the marginal RCA value of tracing.

4.8 Evaluation and cost metrics

Detection is evaluated with precision, recall, F1 and AUC-ROC. F1 is the headline metric because anomaly data are class-imbalanced (normal windows dominate), under which raw accuracy is misleading—a trivial always-normal predictor scores high on accuracy but has zero recall. For the streaming experiments, F1 is computed *prequentially* over the future (R_1 – R_3) windows. Root-cause localisation is measured by top- k accuracy; on the nine-service mesh top- k does *not* saturate without traces (multi-hop latency implicates several ancestors), so the trace-driven lift is visible across k and **top-1 is the strictest measure**.

Because RQ3 reframes the choice of paradigm as an operational decision, we additionally report operational-cost metrics that a static accuracy comparison omits: *maximum per-window latency* (the tail cost, which exposes retraining spikes), *model footprint* in kilobytes, the number of *retained historical windows* (a data-governance and memory cost), and *total CPU time*. These let us characterise the accuracy/cost trade-off rather than accuracy alone.

4.9 Reproducibility

All models are trained and evaluated under a fixed random seed, an identical train/test protocol and the same preprocessing, so cross-cell differences reflect configuration and policy rather than stochastic training variation. The synthetic prequential stream is generated deterministically. All tooling is open-source, and the result datasets (`rq1_completeness`, `rq2_localisation`, `rq3_online_vs_offline`, `rq3_cost`, `rq3_timeline`, `rq4_model_family` and their summaries) underpin every figure and table reported below.

4.10 Threats to validity

Internal validity.

Held-constant controls (model, seed, scenarios, load) and the nested within-subjects design mean cross-cell differences are attributable to the manipulated factors. Fault-injection timing jitter could blur labels; this is mitigated by recording exact start/stop timestamps.

External validity.

A testbed cannot reproduce the full heterogeneity of production [20]. This is mitigated by real OpenTelemetry instrumentation and unmodified backends (so the telemetry schema is production-representative) and by a fault taxonomy drawn from documented real-world failure modes. The nine-service topology, though deeper than many controlled testbeds (depth four, with fan-out and a shared fan-in), is still a single synthetic mesh, which bounds topological generalisation—an acknowledged boundary condition; and the RQ3 stream, while drift-controlled, is synthetic-prequential, so absolute F1 values are testbed-specific even though the *ordering* of paradigms is the result of interest.

Construct validity.

“Detection effectiveness” is multidimensional; using F1, AUC-ROC, latency and cost together avoids over-reliance on a single construct.

Conclusion validity.

A fixed seed and a within-subjects design across all (c, π) cells guard against over-interpreting noise; the RQ3 effect sizes (§5.3) are large relative to any plausible residual variance.

5 Results

We report the four questions in turn. RQ1 and RQ2 characterise what the telemetry makes possible—how completeness affects detection once the stream is non-stationary, and how distributed traces enable root-cause localisation. RQ3 is the article’s headline: it compares the static, periodic and online learning paradigms on the drifting stream and isolates their interaction with completeness. RQ4 then closes the loop, reconciling the offline model-family evidence with the streaming detector that RQ3 shows to be necessary; it is answered last because that implication only becomes legible once the online setting is in view—the strongest *batch* model and the strongest *streaming* model prove not to be the same model.

5.1 RQ1: Does richer telemetry raise detection under non-stationarity?

RQ1 asks whether telemetry completeness improves detection once the system is non-stationary. We therefore report detection F1 by configuration in two settings produced

Table 4 RQ1 — detection by configuration under non-stationarity. F1 on the drifted future stream is for the deployable continual-learning detector (the operative measurement); the held-out slice is a stationary reference. Both rise with completeness; distributed traces are the decisive increment, and more so under drift.

Config	Name	Feat.	F1 (held-out ref.)	F1 (drifted stream)	Δ F1 drifted
C1	Metrics-Only	10	0.8964	0.8130	—
C2	Metrics+Logs	14	0.9153	0.8274	+0.014
C3	+Traces	18	0.9846	0.9737	+0.146
C4	Full MELT	23	0.9855	0.9755	+0.002

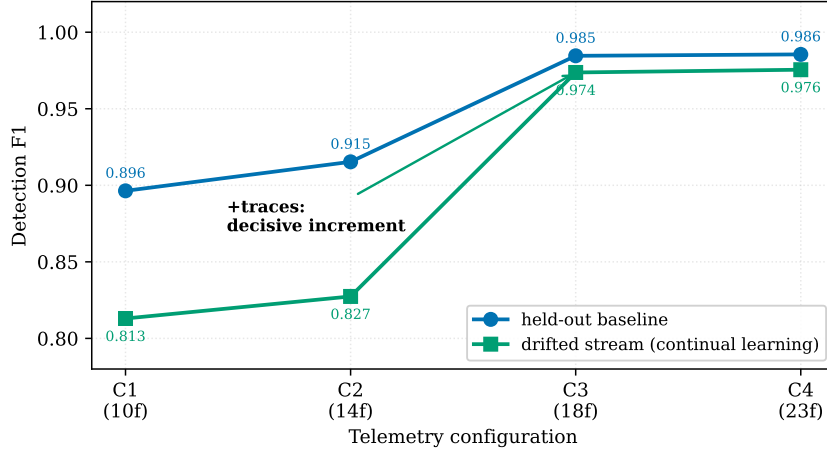


Fig. 7 RQ1 — detection F1 across the four telemetry configurations under non-stationarity: on the held-out reference and on the drifted future stream (continual-learning detector). Completeness improves detection in both settings, and the single decisive increment is the addition of traces (C2→C3), whose effect is larger under drift.

by the *identical* pipeline, varying only the configuration c : on the drifted future stream, for the continual-learning detector one would actually deploy (the operative measurement), and on a controlled, single-distribution held-out slice that serves as a stationary reference (Table 4).

Detection quality rises *monotonically* with telemetry richness in both settings. On the held-out reference, F1 climbs from 0.896 (metrics-only) to 0.986 (full MELT); on the operative drifted stream, the continual-learning detector improves from 0.813 to 0.976. In both, the decisive increment is the addition of distributed traces (C2→C3), and that increment is markedly larger under drift—+0.146 F1 on the drifted stream against +0.069 on the reference—because traces resolve precisely the cross-service signal whose statistics shift most under the load and scale changes that drive the drift. Adding logs (C1→C2) yields a smaller gain in both settings, and adding events and history (C3→C4) leaves detection essentially unchanged.

Table 5 RQ2 — root-cause localisation on the nine-service mesh, traces excluded (C2) vs. included (C3). On a deep graph top- k does not saturate without traces; tracing attains perfect localisation at every k .

Approach	top-1	top-2	top-3
Metrics+Logs (C2)	0.7692	0.8974	0.9487
Metrics+Logs+Traces (C3)	1.0000	1.0000	1.0000

Two conclusions follow. First, completeness matters under non-stationarity, not only on a clean held-out set: richer telemetry raises the attainable detection on a drifting stream, with distributed traces the single most valuable pillar—a more specific finding than the literature’s general “more is better” [9, 11], and one that the near-equivalence of C3 and C4 sharpens further, since for detection the three classical pillars already capture almost all available signal. Second, that this gain materialises at all presupposes a detector that keeps current: whether the telemetry or the *learning paradigm* is the binding constraint under drift—and how the two interact—is precisely the subject of RQ3. **Completeness raises the attainable detection; whether adaptation realises it is the question we reach in RQ3.** We turn first to the localisation contribution of traces (RQ2).

5.2 RQ2: What does distributed tracing add to localisation?

Table 5 reports root-cause localisation with traces excluded (C2) versus included (C3). The introduction of traces raises top-1 accuracy from 0.7692 to a perfect 1.0000, eliminating the localisation errors the metric-and-log approach commits.

The mechanism, as anticipated in the design, lies in the originating-error-span feature. Under the metric-and-log approach a downstream fault raises the latency of *every* service on its call path—in the nine-service mesh a fault at a leaf such as **catalog-service** induces a secondary latency symptom in all of its transitive callers—so the anomaly score implicates as many as four or five ancestors almost as strongly as the true culprit, and a dependent is frequently ranked first. The depth of the graph is what makes this hard: top- k no longer saturates without traces (top-2 = 0.897, top-3 = 0.949), because several plausible culprits must be disambiguated rather than just one of three. The trace feature resolves the ambiguity completely: only the originating service accumulates originating error spans, pulling the true root cause unambiguously to the top and attaining perfect localisation at top-1, and hence at every k . This is precisely the controlled, ablation-style evidence the literature was shown to lack: where Han et al. [11] demonstrate that a fused multimodal architecture beats single-modal baselines, we isolate the trace modality and quantify its marginal contribution while holding all else constant.

5.3 RQ3: Does the learning paradigm matter under drift?

RQ1 and RQ2 measured a controlled, *held-out baseline*—detection accuracy and root-cause localisation for a model trained once on a single distribution. We now re-use the

Table 6 RQ3 headline — F1 on the future (drifted) stream. Best realisable learner in bold.

Config	static	periodic	online	gain vs. static	gain vs. periodic
C1 Metrics-Only	0.3602	0.8204	0.8130	+0.4528	−0.0074
C2 Metrics+Logs	0.3609	0.8323	0.8274	+0.4665	−0.0049
C3 +Traces	0.3701	0.9247	0.9737	+0.6036	+0.0490
C4 Full MELT	0.3705	0.9255	0.9755	+0.6050	+0.0500

identical pipeline and feature sets on the drifted future stream (25,920 post-warm-up windows across R_1 – R_3) and compare the three realisable learning policies, with the all-regimes oracle as a ceiling.

5.3.1 Headline: F1 on the drifted stream

Table 6 and Fig. 8 report F1 over the future stream. The result inverts the baseline picture. The frozen batch model collapses to $F1 \approx 0.36$ *regardless* of how much telemetry it is given: going from metrics-only to full MELT moves its F1 by barely a point ($0.3602 \rightarrow 0.3705$), with its precision pinned near 0.22 throughout. The two adaptive policies recover almost the entire gap, but *which* of them leads depends on telemetry richness. Under thin telemetry (C1, C2) the online and periodic learners are within 0.007 F1 of each other—periodic marginally ahead (0.8204 vs. 0.8130 at C1)—because with few signals the online normaliser has little to re-centre on and much of the drift there is virtual (§5.3.3). Once distributed traces are present (C3, C4) the online-adaptive model pulls clearly ahead, reaching 0.9737 under C3 and 0.9755 under C4 and beating periodic by ~ 0.05 . In every configuration the online model also exceeds the all-regimes oracle (0.8124/0.8166/0.9294/0.9267 for C1–C4; Table 8 note), because the oracle still fits a single static boundary across all regimes whereas the online model tracks each regime as it arrives.

Two features deserve emphasis. First, telemetry richness barely helps the *static* model (the +0.01 from C1 to C4 is within noise) but strongly helps the *online* model (the +0.16 from C1 to C4 closes almost the entire gap to the ceiling). Completeness and adaptation are thus *complementary*: richer signal raises the attainable ceiling, but only adaptation realises it. Second, the online model’s advantage over periodic retraining *grows with richness, crossing from parity to dominance*: from −0.007 at C1, where periodic narrowly leads, to +0.049 at C3 and +0.050 at C4. The more informative the features, the more a continuous learner extracts from each window relative to a lagging batch refit—so distributed traces are precisely what tip the paradigm choice decisively toward online, and under thin telemetry the cheaper periodic schedule remains competitive.

5.3.2 Precision, recall and the stale-normal signature

The F1 headline conceals *how* each policy fails, which Table 7 makes explicit by decomposing the future-stream score into precision, recall and AUC-ROC. The static model exhibits the stale-normal signature in its purest form: its recall is near-perfect

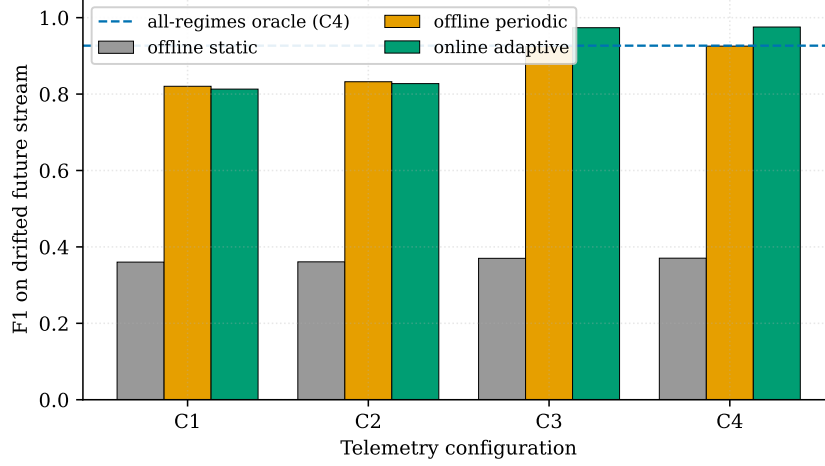


Fig. 8 RQ3 headline — F1 on the drifted future stream by learning policy and configuration. The frozen static model (grey) is flat and low regardless of telemetry; periodic retraining (orange) recovers partially; the online-adaptive model (green) dominates everywhere and, under C3/C4, exceeds the all-regimes oracle (dashed). Contrast with the monotonic baseline ceiling of Fig. 7.

(0.98–1.00) while its precision collapses to ≈ 0.22 at every configuration. That combination is diagnostic—the frozen boundary still fires on the genuine anomalies (high recall) but, because the post-drift normal has moved into the region it labels anomalous, it also fires on healthy traffic, so the overwhelming majority of its alerts are false. No amount of additional telemetry corrects this: the precision is flat across C1–C4 because the failure is in the boundary’s *location*, not in the features’ *informativeness*. The online model shows the opposite profile, trading a little recall for a large precision gain (0.87–0.99), which is the operationally desirable direction given the cost of alert fatigue.

A subtler point hides in the AUC column. The static model’s *thresholded* F1 is flat across configurations, yet its AUC-ROC rises with telemetry, from 0.77 at C1 to 0.86 at C4—the gain concentrated at the trace increment (0.75 at C2 to 0.86 at C3). This means richer telemetry does improve the static model’s *ranking* of windows by anomaly score—traces in particular make anomalous windows more separable—but that improvement is invisible at the fixed operating threshold, because the threshold itself is calibrated to a distribution that no longer holds. In other words, more telemetry sharpens the score but not the stale *cut-point*; only re-learning moves the cut-point. This is a precise, quantitative statement of why completeness and adaptation are complementary rather than substitutable, and it is a distinction that an F1-only analysis would miss entirely.

5.3.3 Per-regime breakdown

Table 8 decomposes F1 by regime, and the decomposition is more revealing than the headline. The online model is best in 8 of the 12 regime×configuration cells; the four exceptions all fall under the thinnest telemetry (C1 and C2), in the *scale-out* (R_2)

Table 7 RQ3 — future-stream precision (P), recall (R) and AUC-ROC by policy and configuration. The static model’s flat, collapsed precision with near-perfect recall is the stale-normal signature.

Config	Policy	P	R	AUC
C1	static	0.221	0.980	0.767
	periodic	0.806	0.836	0.952
	online	0.873	0.761	0.950
C2	static	0.221	0.983	0.754
	periodic	0.813	0.852	0.960
	online	0.878	0.782	0.955
C3	static	0.227	0.999	0.858
	periodic	0.879	0.976	0.991
	online	0.985	0.962	0.998
C4	static	0.227	1.000	0.858
	periodic	0.880	0.976	0.991
	online	0.987	0.964	0.998

and *combined-load* (R_3) regimes, where periodic retraining narrowly leads. This is theoretically coherent rather than anomalous: scale-out, and the volume component of combined load, are predominantly *virtual* drift (§3.2)—the feature baselines shift but the meaning of “anomalous” does not—so a batch refit on recent data recovers the boundary as well as continuous updating, and with thin telemetry the online normaliser has fewer signals to re-centre. By contrast, R_1 *latency regression* is *real* concept drift, and the online model wins it at *every* configuration, the thinnest included; it is also there that the static model is hurt most ($F1 \approx 0.30$, its lowest) while the online model holds 0.79–0.98. Once traces are present (C3, C4) the online model sweeps *all* regimes. The static model’s deficit is thus widest exactly where the drift is conceptual, not merely distributional—the classic stale-normal signature, visible as preserved recall (~ 0.98 –1.00) with collapsed precision (~ 0.22).

5.3.4 Operational cost

Detection quality is only half the decision. Table 9 compares the periodic and online learners on operational cost. The two paradigms have opposite cost profiles (Fig. 9). Periodic retraining produces large per-window *tail* spikes—up to $\sim 1,200$ ms when a refit lands—9–20 $\times$ worse than the online model’s bounded per-window cost (≤ 59 ms). The online model is also ~ 130 –390 $\times$ smaller (≈ 15 KB versus 2–6 MB) and retains *zero* historical windows, against the 2,880-window labelled buffer the periodic model must hold to refit. Its only disadvantage is aggregate CPU: because it updates on every future window rather than performing a handful of batch refits, it spends ~ 3.6 –5 $\times$ the periodic total—but spends it *smoothly*, never stalling the pipeline with a multi-hundred-millisecond spike.

Table 8 RQ3 per-regime F1 by configuration. Best learner per cell in bold. (s = static, p = periodic, o = online.) The all-regimes oracle attains 0.812 (C1), 0.817 (C2), 0.929 (C3), 0.927 (C4); the online model exceeds it at every configuration, decisively so at C3–C4.

Config	Regime	s	p	o
C1	R_1 latency regression	0.3166	0.7627	0.7925
	R_2 scale-out	0.4916	0.8534	0.8251
	R_3 combined load	0.3291	0.8487	0.8202
C2	R_1 latency regression	0.3177	0.7801	0.8060
	R_2 scale-out	0.4908	0.8582	0.8456
	R_3 combined load	0.3295	0.8606	0.8301
C3	R_1 latency regression	0.3018	0.8578	0.9724
	R_2 scale-out	0.5965	0.9676	0.9774
	R_3 combined load	0.3299	0.9533	0.9718
C4	R_1 latency regression	0.3017	0.8587	0.9753
	R_2 scale-out	0.5990	0.9702	0.9789
	R_3 combined load	0.3302	0.9527	0.9731

Table 9 RQ3 cost — periodic vs. online. Latency is the per-window maximum (tail); operational columns are from the cost-profiling run, F1 is the Run-B future-stream F1 of Table 6.

Config	Learner	F1	Max ms/win	Size (KB)	Retained win.	Total s
C1	periodic	0.8204	531.0	5833.3	2880	27.00
	online	0.8130	58.5	14.9	0	136.38
C2	periodic	0.8323	1196.2	5777.5	2880	36.35
	online	0.8274	59.1	15.2	0	129.10
C3	periodic	0.9247	505.4	1990.5	2880	25.26
	online	0.9737	32.1	15.4	0	123.26
C4	periodic	0.9255	508.3	2287.7	2880	24.84
	online	0.9755	34.5	15.7	0	114.25

Table 10 summarises the trade-off as ratios. The online model buys far lower tail latency, a far smaller footprint and zero data-retention requirement—and, once telemetry is rich enough (C3/C4), higher accuracy as well—at the cost of modestly higher steady-state CPU; under thin telemetry it trades a negligible ≤ 0.007 F1 for those same engineering wins. For any deployment with a per-window latency budget—which is to say any real-time detector behind an SLA—the bounded online profile is the safer engineering choice, independent of the accuracy advantage.

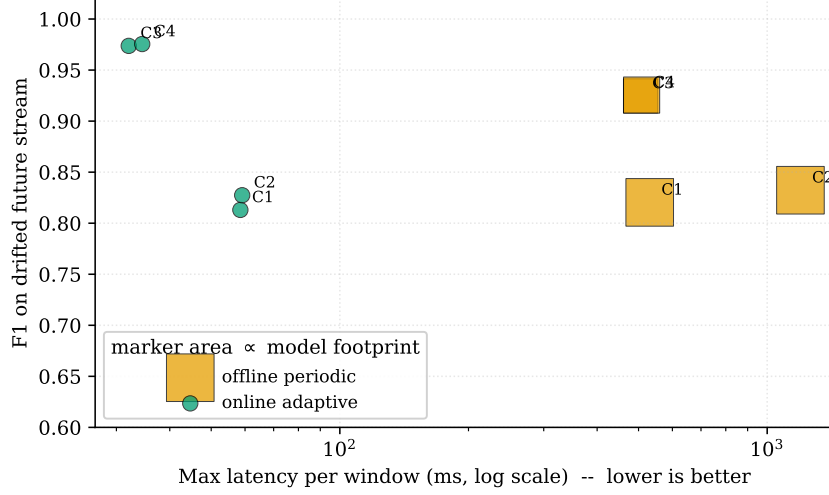


Fig. 9 RQ3 cost/quality trade-off (periodic vs. online). The online learner (green) sits top-left—higher F1 at an order-of-magnitude lower tail latency—with a tiny footprint (marker area $\propto$ model size), while periodic retraining (orange) pays ~ 600 ms refit spikes and a multi-megabyte footprint for lower F1.

Table 10 RQ3 cost ratios (periodic relative to online unless noted).

Config	F1 gain (o-p)	Tail-lat. ratio	Size ratio	Retained (p vs. o)	CPU ratio (o/p)
C1	-0.0074	9.1 $\times$	391.5 $\times$	2880 vs. 0	5.1 $\times$
C2	-0.0049	20.3 $\times$	380.1 $\times$	2880 vs. 0	3.6 $\times$
C3	+0.0490	15.8 $\times$	129.3 $\times$	2880 vs. 0	4.9 $\times$
C4	+0.0500	14.7 $\times$	145.7 $\times$	2880 vs. 0	4.6 $\times$

5.3.5 Adaptation behaviour

Table 11 reports how often the online model re-selects its champion configuration as drift appears. Richer telemetry requires *fewer* corrective adapt events: with thin telemetry (C1) the model switches champions seven times to keep up with drift, three times at C2, but once traces are added (C3/C4) the signal is rich enough that *no* re-selection is needed at all. This is a second face of the completeness $\times$ adaptation interaction: richer signal not only raises accuracy but stabilises the adaptation process, because more of the drift is explained by the EW normaliser and less must be chased by re-selecting the learner.

Table 11 RQ3 online adaptation behaviour by configuration.

Config	Features	Periodic refits	Online adapt events	Final champion
C1	10	51	7	$\eta_0=0.10, \alpha=10^{-3}$
C2	14	51	3	$\eta_0=0.01, \alpha=10^{-4}$
C3	18	51	0	$\eta_0=0.05, \alpha=10^{-4}$
C4	23	51	0	$\eta_0=0.01, \alpha=10^{-4}$

Table 12 RQ4 — detection by model family on full MELT (C4). The three ensemble-tree models lead and are statistically indistinguishable.

Model family	Precision	Recall	F1	Note
Gradient Boosting A	0.995	0.980	0.988	best overall
Random Forest	0.989	0.982	0.986	representative ensemble
Gradient Boosting B	0.989	0.979	0.984	close third
Late fusion	0.999	0.804	0.891	high prec., low recall
LSTM (temporal)	0.165	0.596	0.259	windowed repr. (weak)

5.4 RQ4: Which model family best exploits multimodal telemetry?

We return, last, to a question deferred from the held-out baseline analysis: which model family this pipeline should use, and—now that RQ3 has shown a continuously-adapting detector to be mandatory under drift—what that choice implies for the online learner. The implication only becomes legible once the online setting is in view, which is why this question is answered after, rather than before, the drift comparison.

Table 12 compares five model families on the richest configuration (C4) with identical inputs. Ensemble trees are the most effective family: the three tree models cluster within 0.004 F1 of one another—gradient boosting highest at 0.988, Random Forest at 0.986, and the second boosting implementation at 0.984—each exceeding 0.98 precision, few false alarms, an operationally desirable property given the cost of alert fatigue. The temporal (LSTM) and late-fusion models underperform, instructively rather than disappointingly: the windowed feature representation already captures much per-window signal—and, being assembled from interleaved per-service windows, offers a sequential model little coherent temporal structure—so the LSTM trails badly (0.259 F1); the late-fusion architecture’s requirement of cross-pillar agreement suppresses false positives at the cost of recall (0.999 precision against 0.804 recall).

The lesson, on which the drift comparison of RQ3 has already drawn, is that the benefit of rich multimodal data is conditioned on the model’s capacity to exploit it: a powerful, data-hungry architecture does not automatically beat a simpler ensemble when the feature representation is already informative. This is why the offline learners in RQ3 used a Random Forest, and why the online learner is a lightweight linear model—on tabular telemetry, simplicity buys both accuracy and the cheap, bounded per-window update that streaming requires.

This is the apparent tension the article’s ordering was designed to resolve: if ensemble trees are the strongest *batch* family, why is the *online* learner—now shown to be the one that matters under drift—not also a tree? The answer is that the two settings reward different properties. A Random Forest is an excellent *static* estimator but a poor *incremental* one: it cannot be updated by a single example without either refitting (the expensive, bursty periodic path) or resorting to incremental-tree variants whose drift behaviour in the upper splits is fragile. The online setting prizes a bounded, smooth per-window update and graceful adaptation over raw stationary accuracy, and a normalised linear model delivers exactly that—its per-update cost is $O(d_c)$, its footprint is kilobytes, and, crucially, the configuration-aware features (§4.5) have already done the non-linear work of summarising raw telemetry into informative aggregates, leaving a decision surface that is close to linearly separable. The RQ3 results already reported vindicate the choice: once the telemetry is rich, the linear online model beats both batch paradigms on the drifted stream and, under full telemetry, exceeds the Random-Forest oracle itself (Table 6). The strongest *static* model and the strongest *streaming* model are simply not the same model, and conflating the two is one way the single-snapshot literature is misled.

6 Discussion

6.1 Drift, not completeness, is the deciding factor

The juxtaposition of RQ1 and RQ3 is the paper’s central message. On the held-out baseline, completeness is decisive and the model is excellent regardless of paradigm: F1 climbs smoothly to 0.986 (Table 4). On a drifted stream the same completeness barely moves the static model (Table 6); what determines whether the detector works is the learning policy. The static model’s collapse is not a tuning failure that more telemetry or a better threshold could fix—it is structural. Once operations move the baseline, a boundary fitted to the old normal necessarily mislabels the new normal, and no amount of additional signal about the old normal helps. This reframes the field’s guiding question. “Does observability matter?” has the answer “yes, for the ceiling”; but “what makes a detector work in production?” has the answer “keeping it current”. The two are complementary, and a study that varies only one axis can reach a misleading conclusion about the other.

6.2 The completeness×adaptation interaction

The two axes are not independent. Three results show their interaction. First, richness helps the online model far more than the static one (+0.16 vs. +0.01 from C1 to C4), so the benefit of instrumentation is itself *contingent on adaptation*: a practitioner who invests in tracing but freezes the model captures almost none of the value (compare static C4 = 0.37 with online C4 = 0.98). Second, the online advantage over periodic *grows with richness, crossing from parity to dominance*: the two are within 0.007 F1 under thin telemetry (periodic marginally ahead), but the online model leads by ~ 0.05 once traces are present, because a continuous learner extracts more per window when

each window carries more signal. Third, richer telemetry stabilises adaptation, reducing champion re-selections to zero once traces are present (Table 11). The practical reading is that observability investment and continual-learning investment should be made *together*; either alone is sub-optimal, and the joint return exceeds the sum of the parts.

6.3 Operational implications and an SLA framing

Because RQ3 measures cost as well as accuracy, the choice of paradigm can be posed as a service-level decision. Periodic retraining, the field’s usual concession to drift, is revealed as a partial, expensive and laggy patch: it lags by up to a retraining interval, must store a large labelled buffer (a data-governance liability as well as a memory cost), and stalls the pipeline with ~ 600 ms refit spikes that, behind a real-time SLA, are themselves an incident. The online model’s per-window cost is bounded and small; its footprint is kilobytes; it retains no raw data, which simplifies privacy and retention compliance. Its only cost is higher aggregate CPU, spent smoothly. For a detector that must respond within a per-window budget, the bounded-latency profile is decisive on engineering grounds even before the accuracy advantage is considered. This connects directly to the practitioner guidance that observability is an exercise in managing cost as much as capability [17, 38].

The cost asymmetry, moreover, *compounds* with scale rather than washing out. Consider the operational picture as the deployment grows from a handful of services to a large mesh. The periodic model’s refit spike grows with the size of its training buffer and feature space, and its labelled-buffer footprint grows with both retention horizon and telemetry richness; every monitored service multiplies these costs, and each refit remains a synchronous stall that the surrounding pipeline must absorb. The online model’s per-window update, by contrast, stays $O(d_c)$ and kilobyte-sized *per detector* regardless of how long the stream has run, because it retains no history—the property that makes it cheap is structural, not a small-scale artefact. A fleet of online detectors therefore presents a flat, predictable resource profile that an operator can provision for, whereas a fleet of periodic detectors presents a sawtooth of refit spikes whose worst case is what capacity planning must target. In a production setting governed by tail latency and by the cost of over-provisioning for peaks, the bounded online profile is not merely preferable but materially cheaper to operate at scale, reinforcing the accuracy case rather than trading against it.

6.4 When is periodic retraining acceptable?

The four cells in which periodic retraining edges the online model (scale-out and combined-load under C1 and C2; Table 8)—enough to give periodic the aggregate lead at thin telemetry—are not a defect of the experiment but a boundary condition worth stating precisely, because it tells a practitioner when the cheaper, more familiar periodic approach suffices. Periodic retraining is competitive exactly when three conditions coincide: the drift is predominantly *virtual* (the feature distribution moves but the concept does not), so that a refit on recent data restores the boundary without needing to relearn the decision; the telemetry is *thin*, so that the online normaliser has

few signals from which to re-centre continuously and its advantage is muted; and the operating environment can *tolerate the tail-latency spikes and the labelled-buffer footprint* that periodic refitting imposes. The scale-out and volume-driven regimes under metrics-only telemetry satisfy all three, and there periodic wins by a slim margin (a few thousandths to ~ 0.03 F1). The moment any of the three fails—real concept drift (latency regression, R_1), richer telemetry (C3/C4), or a strict per-window latency budget—the online model regains a decisive and widening lead. The practical implication is not “online always”, but “online unless your drift is purely cosmetic, your telemetry is minimal, and your latency budget is loose”—a narrow and identifiable corner of the design space. For the common case of a richly-instrumented service subject to deployments (real drift) behind an SLA, online adaptation is the dominant choice on every axis we measured.

6.5 Practical guidance

The results support concrete guidance.

- **Minimum-viable observability for detection** is metrics+logs+traces (C3). Traces are the single highest-value increment for both detection (RQ1) and localisation (RQ2), and events/history add little to detection (C3 $\approx$ C4). A team constrained on instrumentation budget should prioritise tracing over event collection for the detection-and-localisation use case.
- **Continual learning is mandatory, not optional**, on any stream subject to deployments, autoscaling or load shifts—that is, essentially all production cloud-native systems. A frozen model is unsafe under drift irrespective of its telemetry (RQ3).
- **Prefer online to periodic** where a label signal is available and a per-window latency budget exists. Periodic retraining is acceptable only where labels arrive in large delayed batches and tail-latency spikes are tolerable.
- **Invest in observability and adaptation together**. The joint return exceeds either alone (§6.2).

These points compose into a short deployment checklist. *(i)* Instrument to at least the three-pillar level (C3); budget for trace sampling rather than omitting traces, since traces are the decisive signal for both detection and localisation. *(ii)* Choose the learning paradigm by interrogating the drift, not the model: if deployments and autoscaling routinely shift the baseline—real concept drift—a frozen model is disqualified regardless of its validation score. *(iii)* Default to online adaptation with a bounded per-window update and a running normaliser; reserve periodic retraining for the narrow corner identified in §6.4. *(iv)* Monitor the detector itself: track precision over time, because a precision decline with stable recall is the early signature of stale-normal drift (§5.3.2) and a trigger to verify that adaptation is keeping pace. *(v)* Treat the labelled-data buffer of any periodic scheme as a governance liability, not just a memory cost, and prefer the zero-retention online design where data-handling constraints apply.

6.6 Analysis by fault category

The aggregate metrics conceal an instructive variation across the fault taxonomy that explains *why* each pillar contributes what it does. Resource-exhaustion faults—memory leaks and CPU saturation—are largely metric-visible: they move CPU and memory utilisation directly, and even the metrics-only configuration detects them well, which is why C1 already attains $F1 \approx 0.91$ on the held-out baseline set. Their localisation is the harder part, because resource pressure at one pod induces latency at its callers, smearing the metric signal across the dependency chain—precisely the ambiguity that the trace originating-error-span feature resolves (RQ2). Performance-degradation faults—latency spikes and error bursts—are where logs and traces earn their keep: an error burst raises the error-log count (a C2 feature) before it is unambiguous in the latency percentiles, and a latency spike that propagates is disentangled by span-level timing (a C3 feature). Availability faults—OOMKilled and CrashLoopBackOff—are the ones that motivate the events pillar: they manifest as discrete Kubernetes lifecycle events that no metric or log aggregate captures as cleanly, which is why C4 retains a localisation and robustness edge even though its detection F1 is statistically tied with C3 on the held-out baseline set. The picture that emerges is that the pillars are not redundant but *specialised*: each is the decisive signal for a different region of the fault taxonomy, and full MELT wins not by being uniformly better but by covering the union.

Under drift this specialisation interacts with the learning policy, and the per-regime breakdown (Table 8) separates the two kinds of drift cleanly. The *latency-regression* regime R_1 is real concept drift—a latency level that was anomalous becomes the new normal—and it is there that the static model collapses hardest ($F1 \approx 0.30$, its lowest at every configuration); richer telemetry does not rescue it, because the boundary, not merely the scaling, is now wrong. The *scale-out* regime R_2 is predominantly virtual drift—the resource-utilisation baselines move but “anomalous” still means the same thing—and here the static model fares best of the three regimes ($F1$ up to 0.60) and periodic refit even edges the online model under thin telemetry, because re-fitting on recent data restores the shifted scaling that a batch model needs. The *combined-load* regime R_3 superimposes both, and under thin telemetry periodic edges the online model here too. Only the online normaliser, which re-centres every feature continuously, neutralises the virtual component while incremental fitting tracks the conceptual one, which is why the online model holds $F1 \approx 0.79\text{--}0.98$ across $R_1\text{--}R_3$ where the static model never exceeds 0.60 and bottoms at 0.30.

6.7 Relation to prior work

Our held-out baseline findings are consistent with, and sharpen, the literature. The monotonic completeness benefit echoes the general multimodal advantage reported by Han et al. [11] and the multi-source gains of Chen et al. [13] and Wang et al. [12], but—by holding the model fixed and varying only the signal set—attributes the largest single increment specifically to traces, which a fused-architecture comparison cannot do. The strength of ensemble trees on multimodal tabular telemetry (RQ4) corroborates the classical-ML results of Lomio et al. [9] and extends them to the

multimodal setting, while clarifying why a deep temporal model need not win when the windowed representation is already informative—consistent with the accuracy–cost concerns around VAE/temporal approaches such as Night’s Watch [10].

The drift findings (RQ3) speak to gaps the surveys identify but do not close. Raeiszadeh et al. [14] name adaptability as a systematically unmet requirement; we quantify exactly what its absence costs (a collapse from 0.98 to 0.36) and what meeting it buys. Hrusto et al. [20] warn that single-snapshot, synthetic-distribution evaluation may not transfer; our two-axis design shows precisely *how* it fails to transfer—the held-out baseline ranking does not predict the drifted ranking—and offers a controlled-drift testbed as a middle path between synthetic-benchmark and uncontrolled-production evaluation. The industrial inadequacy of static thresholds reported by Islam et al. [4] is reproduced here as the stale-normal collapse and given a remedy. Finally, the proactive, observability-integrated framing of Mart et al. [5] is extended from *forecasting* ahead of failure to *continually re-learning* what failure looks like as the system changes.

7 Threats to Validity

The principal external-validity caveat is that the RQ3 stream, though drift-controlled and built on production-representative telemetry, is synthetic-prequential; absolute F1 values are testbed-specific. We stress that the contribution is the *ordering and its reversal*, not the absolute numbers: that the baseline-best ranking does not survive drift, and that online dominates static at every cell and leads periodic in 8 of 12 regime×configuration cells (the four exceptions being virtual-drift scale-out and combined-load under thin telemetry, where the cheaper periodic schedule is competitive), is robust to the absolute scale. The nine-service topology, though deeper than many controlled testbeds, is still a single synthetic mesh, which bounds topological generalisation; on it top-*k* remains informative even without traces (Table 5). The online model depends on a usable label signal; where labels are scarce or heavily delayed, semi-supervised or unsupervised drift-adaptation would be required—a direction for future work. Finally, a single seed underlies the reported run; the effect sizes are large relative to plausible seed variance, but a multi-seed study would tighten the conclusion-validity argument.

The streaming and LLM additions (§4.3) are reported conservatively for the same reason. The RQ3 numbers come from the deterministic in-process prequential harness, not a loaded Kafka broker, so broker-induced tail latency, consumer lag and loss-on-rebalance are out of scope; they would, if anything, only tighten the per-window latency budget the online model already meets, and a broker is additionally a shared dependency a deployment must size for. The served Qwen2.5-3B detector is implemented but not yet evaluated at scale—we therefore make no accuracy claim for it, and the clearly-marked heuristic fallback is a sanity scaffold, not a proxy for the model. Because that detector consumes *raw*, attacker-influenceable log and trace text, it also carries an indirect prompt-injection exposure [39] that the engineered-feature classifiers, which read only numeric aggregates, do not; treating telemetry text as untrusted before it reaches the prompt is future work.

It is worth being explicit about what a full real-deployment validation would add, and why we regard the present controlled study as the necessary first step rather than a substitute. On a live production stream, drift is neither scheduled nor labelled: regime changes arrive unannounced and entangled, ground truth is partial and delayed (often reconstructed after the fact from incident timelines), and the fault distribution is long-tailed in ways no injection taxonomy fully reproduces. A field study would therefore measure something our testbed cannot—robustness to *unknown* drift and to *imperfect* labels—but at the price of the very thing that makes our central claim interpretable: the ability to attribute the reversal between the held-out baseline and drifted rankings to drift alone, with the model, features, faults and seed held fixed. The two evidence types are complementary, and in the order presented: the controlled result establishes *that* the learning paradigm dominates completeness under drift and *why*; a subsequent field deployment would establish *how much* of that advantage survives the additional noise of production. We read the present findings as a strong prior for that field study, not as a replacement for it, consistent with the synthetic-versus-operational tension that motivated the design [4, 20].

8 Related Work

Having presented our results, we now situate the study within the literature, at the intersection of four areas: observability in cloud-native systems (§8.1), machine-learning anomaly detection (§8.3), observability-driven root-cause analysis (§8.4), and—the axis the present work foregrounds—concept drift and online learning over telemetry streams (§8.5). Section 8.6 considers the persistent divide between synthetic and operational evaluation, and §8.7 synthesises the gap.

8.1 Observability in cloud-native systems

The term *observability* is borrowed from control theory, where a system is observable if its internal state can be reconstructed in finite time from its outputs [6]. Transposed to distributed software, the framing is apt: a microservice mesh is too large, too dynamic and too ephemeral to inspect directly, so its state must be *inferred* from emitted telemetry. Kosińska et al. [6], in a systematic mapping of 56 primary studies, establish that cloud-native observability is operationalised through three principal domains—monitoring (metrics), logging, and distributed tracing—and identify a research imbalance in which monitoring receives disproportionate attention relative to logging and tracing. That imbalance matters here because it foreshadows an evaluation imbalance: if most detectors are built and tested on metrics alone, the marginal value of logs and traces is left empirically uncharacterised. More recent surveys confirm and extend this picture for microservice systems specifically, while underlining how quickly the tooling and the threat surface continue to move [40].

The practitioner literature crystallises the same three pillars into the MELT model and an end-to-end pipeline: instrumented applications emit telemetry, a vendor-neutral collector (OpenTelemetry) normalises it, backends (Prometheus for metrics, Loki for logs, Tempo for traces) store and index it, and analysis tools query it [7, 8]. The introductory and practice-oriented guides emphasise two themes that bear directly

on this paper. First, observability is sold as the foundation for *automated* analysis—alerting, anomaly detection and self-healing—rather than human dashboard-watching [38, 41]. Second, observability is repeatedly described as a *cost* as much as a capability: instrumentation overhead, data-engineering load, and the storage of high-cardinality telemetry are real expenses that a leading practice must manage deliberately [6, 17]. The economic premise of our RQ1 follows from this: if each additional pillar carries a cost, practitioners need evidence on its detection benefit.

8.2 The observability pipeline and its tooling

Understanding the cost and the structure of the telemetry on which our detectors operate requires a brief account of the modern observability pipeline, which the practitioner literature documents in depth [7, 8, 38, 41]. The contemporary reference design is built around *OpenTelemetry*, an open standard and set of SDKs that has become the de facto vendor-neutral instrumentation layer [7, 42]. OpenTelemetry defines a common data model for the three pillars—a metric is a numerical measurement with a timestamp and a set of attributes; a log record is a timestamped, optionally structured event; a span is a timed unit of work within a trace, carrying a trace identifier, a parent identifier, and attributes—and a *collector* that receives, processes and exports this data to backends. The significance of this standardisation for the present study is that the telemetry schema emitted by an OpenTelemetry-instrumented application is the same whether it is generated by a small testbed or a production fleet; building our testbed on OpenTelemetry therefore directly addresses the external-validity concern that synthetic data may not resemble operational data [20].

Boten [7] emphasises that instrumentation is the part of observability with the highest leverage and the highest cost: auto-instrumentation agents capture a baseline of metrics, logs and traces without code changes, but rich, semantically meaningful telemetry—custom metrics, structured log attributes, business-relevant span attributes—requires deliberate engineering. Crucially for our cost argument, tracing is the most expensive pillar to operate. Every request generates a tree of spans; at high request rates this volume is unsustainable to store in full, so production deployments apply *sampling*—head-based (decide at trace start) or tail-based (decide once the trace completes, retaining error traces preferentially). Sampling trades completeness for cost, and the choice of sampling strategy materially affects what trace-derived features a detector can compute; an active research line tries to soften this trade-off with smarter, code- or commonality-aware sampling [19, 43], or to recover traces without instrumentation at all [44], and the broader point that observability spend must be justified by return is now made explicitly [18]. This is one concrete reason that the marginal detection value of traces (our RQ1, RQ2) is a question with real economic stakes rather than a foregone conclusion.

On the storage and query side, Chapman and Holmes [8] document the now-common “LGTM”-style stack—Loki for logs, Grafana for visualisation, Tempo for traces, and a Prometheus-compatible store for metrics—which is exactly the back-end topology our testbed adopts (§4.2). They stress two operational realities that bear on machine-learning consumers of this data. The first is *cardinality*: metrics are indexed by label sets, and high-cardinality labels (per-user, per-request identifiers)

cause a combinatorial explosion in time-series count that dominates storage cost and query latency; feature engineering for detection must therefore aggregate over high-cardinality dimensions rather than consume them raw. The second is the *correlation problem*: metrics, logs and traces are emitted by separate subsystems with separate indices, and joining them—e.g. pivoting from an anomalous metric to the trace that explains it—is the central workflow of an investigation and a non-trivial engineering task. A detector that fuses the pillars, as ours does through configuration-aware feature extraction (§4.5), performs a form of this correlation automatically, which is part of what makes multimodal detection valuable.

The industry guides [17, 38, 41] frame observability less as a tooling choice than as a practice with a maturity curve. A team typically begins with metrics and dashboards (reactive monitoring), adds structured logging, then distributed tracing, and finally invests in correlation, service-level objectives and automated analysis. This maturity progression is, in effect, the completeness axis of our RQ1 made organisational: C1→C4 traces the same path a team walks as it matures. The guides are also candid that each step up the curve adds cost—more storage, more instrumentation effort, more cognitive load—and that the return on each step is not self-evident [17]. That candour is precisely the gap our controlled completeness study is designed to fill: to replace the maturity-curve intuition that “more is better” with a measured marginal value for each pillar.

8.3 Anomaly-detection techniques

Raeiszadeh et al. [14] provide the most current taxonomy of ML-based anomaly detection for cloud-native architectures and, valuably, evaluate the field against six requirements: adaptability to dynamic topologies, distributed operation, interpretability, scalability with high-velocity telemetry, real-time detection latency, and cross-layer telemetry integration. Their central finding is that existing approaches systematically fail on adaptability, distributed operation and interpretability. We adopt their requirements lens as an evaluative yardstick, and note that *adaptability*—the ability to track a moving distribution—is exactly the requirement a static evaluation cannot test and that RQ3 is designed to probe.

Classical machine learning.

Traditional algorithms—Support Vector Machines, Isolation Forest, Random Forest and gradient-boosted trees—remain competitive for metric-based detection. Lomio et al. [9] train ML models on 168 runtime metrics from Kafka and Zookeeper, reporting AUC above 95% for performance anomalies with training times of 8–17 minutes. The result is a useful baseline but illustrates the field’s characteristic limitations: the evaluation is single-modality and batch, a model trained once on a static snapshot. Neither property survives contact with a streaming, multimodal, non-stationary environment—the point RQ3 makes empirical.

Deep and temporal models.

Deep learning attracts interest for its capacity to model temporal and spatial structure. Dkma et al. [10] introduce the *Night’s Watch* algorithm, a variational-autoencoder

architecture integrating multi-source data with temporal features, reporting precision up to 92% in microservice anomaly detection. Recurrent architectures (LSTM, GRU) capture temporal dependencies in metric streams, and graph neural networks encode service-topology relations. Across this sub-field an *accuracy–interpretability–cost* trilemma recurs: deep models report strong precision but are opaque (violating the interpretability requirement of [14]), are expensive to train, and frequently require the very multimodal data whose marginal value has not been isolated. The literature thus offers increasingly sophisticated *models* while leaving the prior question—how much *signal* they actually need, and whether they can be *kept current*—unanswered.

The recent surge, by modality.

Since then the field has fragmented along the three telemetry pillars, and a brief tour explains why a study that fixes the model and varies the *signal* is overdue. For metrics, deep models for multivariate time series now dominate; recent surveys catalogue an unwieldy zoo of reconstruction-, forecasting-, attention- and contrastive-learning architectures [27, 45, 46], and even the prior question of which metrics actually carry the signal is now studied in its own right [47]—even as pointed critiques argue that much of the reported progress is an artefact of flawed evaluation protocols rather than genuine gains [16, 48]. For logs, the trajectory runs from template-sequence models to pre-trained transformers and, most recently, large language models that read raw log text [15, 30, 49]. For traces, graph deep learning has become the default, encoding the call graph and its span timings—often fused with metric channels—to surface structural anomalies [28, 29, 50, 51]. Each pillar thus has its own thriving sub-literature and its own state of the art, yet the three are seldom measured against one another on the same system—precisely the comparison RQ1 makes.

Multimodal fusion and the LLM turn.

A newer strand fuses the pillars outright. Contrastive and diffusion-based architectures learn a joint representation of metrics, logs and traces and report gains over any single modality [32, 52], and large multimodal models have begun to serve as few-shot anomaly analysers that ingest heterogeneous signals directly [31]. Beneath all of this sits the broader programme of *AI Ops*—machine learning applied to IT operations—whose recent surveys trace the same arc from bespoke models towards general-purpose, LLM-driven pipelines [53]. Two features of this literature matter here. First, it almost uniformly assumes that more signal is better, without isolating the marginal contribution of each pillar. Second, and more consequentially, it is overwhelmingly *batch*: a model is trained once and scored on a held-out slice of the same distribution, leaving the question of how it behaves once that distribution moves—the subject of RQ3—largely untouched. A few works lean the other way, proposing forecasting- or online-flavoured detectors [33, 54], and we return to them in §8.5.

Observability-integrated, proactive detection.

Mart et al. [5] address anomaly detection in Kubernetes clusters using Prometheus metrics, observing that most monitoring remains *reactive*, triggering only once defects

are imminent, and proposing time-series forecasting for earlier detection. Their contribution is conceptual as much as technical: it reframes detection as a proactive, observability-integrated capability rather than a threshold alarm—a framing this paper inherits and extends from proactive to *continually self-correcting*.

8.4 Root-cause analysis through observability data

Detection is rarely an end in itself; it serves root-cause analysis (RCA), whose effectiveness is tightly coupled to observability quality. Han et al. [11] propose *HolisticRCA*, formalising RCA along three dimensions—resource-entity localisation (*which* service failed), observability-feature identification (*which* signal indicates it), and fault-type classification (*what kind* of failure)—and using a graph attention network to map heterogeneous trace, metric and log features into a shared space. Their most consequential finding for us is that single-modal RCA is internally inconsistent: a metric-based localiser may implicate one entity while a log-based classifier indicates a different fault type, and jointly modelling all modalities resolves the inconsistency. This is the strongest existing evidence that modalities are *complementary* rather than redundant—but it is demonstrated through a single fused architecture, not a controlled ablation, so the marginal contribution of traces specifically remains entangled with model design. RQ2 isolates it.

Complementary approaches reinforce the value of dependency structure. Wang et al. [12] build Bayesian networks from performance metrics (CloudRanger), showing that service-dependency graphs derived from observability data improve fault localisation. Chen et al. [13] extend multi-source detection to end-to-end service-function chains (cSFCAD), integrating data-plane and control-plane signals across virtualised network functions. Pedroso et al. [37] combine LLM-based log analysis with Bayesian networks for probabilistic root-cause inference, demonstrating that language models can extract structure from heterogeneous event data, and—relevant to our fault-injection design—ground their evaluation in controlled fault injection with recorded timestamps. Together these works establish that topology and multi-source fusion improve RCA, yet none varies observability completeness as a controlled factor, and none addresses the temporal validity of the learned model.

The RCA literature has since accelerated sharply—by 2024 a survey already needed to taxonomise dozens of methods [55]. Its centre of gravity has moved towards causal inference and online change-point reasoning, in which a causal graph over services and metrics is constructed and the fault traced back to its source [56–58]; a careful benchmark of twenty-one such methods found that the strongest perform well even on short metric windows, but that the ranking is highly method- and dataset-dependent [56]. In parallel, large language models have entered RCA both as cloud-incident diagnosticians that read mixed telemetry alongside human context [59] and as reinforcement-tuned failure localisers [60]. This very activity has exposed how fragile cross-paper comparison remains, prompting dedicated benchmarks built on standardised multimodal telemetry [36]. None of it, though, unsettles the two observations above: completeness is not the controlled variable, and the learned model is assumed to remain valid as the system evolves.

8.5 Concept drift and online learning

The variable this paper foregrounds is largely absent from the cloud-native anomaly-detection literature but central to the streaming-learning literature, where it has accumulated its own substantial body of surveys [25, 26, 61]. *Concept drift* denotes a change over time in the joint distribution $P(\mathbf{x}, y)$ of features $\mathbf{x}$ and labels y . In an observability stream, drift is the norm, not the exception: a deployment changes $P(\mathbf{x} \mid y=\text{normal})$ (the same healthy behaviour now produces different metric values), an autoscaling event shifts resource-utilisation baselines, and a traffic surge moves latency and throughput distributions simultaneously. A detector trained on yesterday’s normal will, after such a shift, assign high anomaly scores to today’s perfectly healthy traffic—a failure mode we term *stale-normal*.

Three families of response exist. A *static* model is trained once and frozen; it is cheap and predictable but, by construction, cannot track drift. A *periodically-retrained* model is refitted at intervals on a sliding window of recent labelled data; it tracks drift in discrete steps but lags by up to one retraining interval, must retain a labelled buffer, and incurs a bursty refit cost. An *online* (incremental) model updates its parameters on every observation via partial fitting; it tracks drift continuously, retains no raw history, and spreads its cost smoothly, at the price of higher aggregate update work and a dependence on a usable label signal. The industrial-scale experience of Islam et al. [4] and the adaptability deficit identified by Raeiszadeh et al. [14] both point to continual learning as the missing ingredient, but neither the surveys nor the primary studies reviewed above evaluate the three paradigms head-to-head on the same observability stream. That head-to-head comparison is RQ3.

The broader stream-learning literature offers a spectrum of online algorithms, and it is worth situating our choice within it. At one end sit incrementally-updated linear models—stochastic-gradient classifiers of the kind our online detector uses—which are cheap, have an $O(d)$ per-update cost, and degrade gracefully, but assume a (locally) linear decision surface. In the middle are incremental decision trees such as the Hoeffding tree and its variants, which grow a tree online using statistical bounds to decide splits; they capture non-linear structure but update less cheaply and are more sensitive to drift in their upper splits. At the adaptive end are explicit drift-detection wrappers (for example windowed change detectors that resize their memory as the stream’s error rate shifts) and streaming ensembles that retire and spawn base learners as concepts change. A growing line of work applies exactly these ideas to anomaly detection—incremental autoencoders that re-fit as the stream evolves [34], dual drift-detection schemes for anomalous sequences [62], and online learners that are shown to beat their batch counterparts once drift is present [33]; the MLOps community frames the same problem as *model decay* and treats its detection as a first-class monitoring task [63]. Two design lessons from this literature inform our system. First, *normalisation under drift is as important as the learner*: a model that does not re-centre its inputs will mistake a baseline shift for signal, which is why our detector pairs the classifier with a running normaliser rather than relying on the classifier alone. Second, *adaptation can be implicit or explicit*: our champion bandit is an implicit, lightweight form of the ensemble idea, re-selecting among a small pool by recent performance rather than maintaining a large, costly ensemble. The contribution of RQ3 is not a new

online algorithm but a controlled measurement of what *any* reasonable online learner buys, and at what operational cost, relative to the batch paradigms that dominate the cloud-native literature.

8.6 Monitoring data and real-world evaluation

Hrusto et al. [20] provide the field’s most recent systematic mapping (104 papers) focused on anomaly detection evaluated with *real-world* monitoring data. Their central argument is methodological: synthetic datasets do not adequately represent the complexity of operational cloud environments, and results obtained on them may not transfer. They taxonomise monitoring data by structure, type and origin, mapping each to suitable preprocessing and detection techniques. Islam et al. [4] corroborate this from industrial practice at IBM Cloud scale, finding threshold-based monitoring inadequate for the heterogeneity and scale of real environments—a conclusion their 2025 follow-up reinforces while releasing a public industrial dataset [64]. The community has answered the transferability problem on two fronts. One is a wave of openly released microservice datasets and benchmarks that carry labelled faults across several telemetry modalities [35, 36, 65], frequently generated through chaos-engineering fault injection whose practice is now mature enough to warrant its own systematic review [66]. The other is a sharper scrutiny of evaluation itself: recent work shows that popular anomaly-detection metrics can reward near-random predictions and that headline gains often evaporate under stricter protocols [16, 48], a caution we take seriously in §4.8.

This exposes a methodological double bind. Realistic non-stationarity argues for production data, yet *controlled* manipulation of observability completeness *and* drift is only cleanly achievable when the data-generating process is itself controlled—i.e. in a testbed with deliberate fault injection and deliberate regime changes. We resolve the bind, as §4 details, by using a production-representative testbed (real OpenTelemetry instrumentation and real Prometheus/Loki/Tempo backends) under controlled fault injection *and* controlled drift, rather than either a purely synthetic benchmark or uncontrolled production logs.

8.7 Synthesis and research gap

Four convergent gaps emerge.

Gap 1 — Completeness is rarely the controlled variable. Most techniques are evaluated on a single modality [5, 9], while the strongest multimodal evidence [11] varies the *model* rather than the *signal set*. Few studies hold the model fixed and systematically vary completeness to measure the marginal value of each pillar.

Gap 2 — The learning paradigm is almost never a variable. Evaluation is overwhelmingly single-snapshot batch [9–11]. The static, periodic and online paradigms are not compared head-to-head on a common observability stream, despite adaptability being a named, unmet requirement [14].

Gap 3 — The synthetic-versus-operational divide. Evaluation is typically either synthetic-and-controlled or operational-and-uncontrolled, rarely both [4, 20].

Table 13 Positioning against representative prior work. ✓ = addressed; **part** = partially or implicitly; — = not addressed.

Study	Completeness	Paradigm	Drift	RCA	Real telemetry
Lomio et al. [9]	—	—	—	—	✓
Mart et al. [5]	—	part	—	—	✓
Dkmak et al. [10]	part	—	—	part	✓
Han et al. [11]	part	—	—	✓	✓
Wang et al. [12]	—	—	—	✓	✓
Chen et al. [13]	part	—	—	✓	✓
Pedroso et al. [37]	part	—	—	✓	✓
Islam et al. [4]	—	part	part	—	✓
This work	✓	✓	✓	✓	✓

Gap 4 — Minimum-viable observability and when to adapt. There is little practical guidance on either question. Practitioners face a real instrumentation cost [6, 17] and a real drift problem [4] with scant evidence to guide either decision.

Table 13 positions the present work against the most directly comparable studies along five axes: whether telemetry completeness is varied as a controlled factor, whether the learning paradigm is varied, whether drift/non-stationarity is evaluated, whether root-cause localisation is addressed, and whether the telemetry is production-representative. No prior study varies both the signal set and the learning paradigm, and none evaluates the three paradigms head-to-head under controlled drift.

This paper addresses Gap 2 as its primary target (RQ3) and Gaps 1, 3 and 4 as the controlled foundation that makes the RQ3 result interpretable. The design fixes the ML pipeline and varies, orthogonally, the signal set (RQ1–RQ2) and the learning paradigm under drift (RQ3) on a production-representative, fault-injected, drift-controlled testbed.

9 Conclusion and Future Work

This paper asked whether, for a non-stationary observability stream, the learning paradigm matters more than telemetry richness, and how the two interact. By crossing four nested telemetry configurations with three learning policies on a single production-representative, fault-injected, drift-controlled testbed, we reached a two-part answer. On the held-out baseline, completeness is decisive: detection F1 rises monotonically from 0.896 to 0.986, traces are the single most valuable increment, ensemble trees are the strongest family, and traces lift root-cause top-1 localisation from 0.77 to a perfect 1.0 on a deep nine-service mesh. But under drift this ceiling does not survive a frozen model: the static detector collapses to $F1 \approx 0.36$ regardless of telemetry, periodic retraining recovers much of the gap but expensively, and a lightweight online-adaptive detector dominates once telemetry is rich—decisively at C3/C4 and across two-thirds of regimes, reaching ≈ 0.98 under full MELT and beating an all-regimes oracle at every configuration—while being 9–20× better on tail latency, ~ 130 – $390\times$ smaller, and free of any data-retention requirement.

The synthesis is that telemetry completeness and continual learning are complementary, not substitutable. Richer signals raise the attainable ceiling and stabilise adaptation; only adaptation realises that ceiling on a live, drifting system. The field’s guiding question, “does observability matter?”, is therefore incomplete: observability matters for what is *achievable*, but keeping the model current is what makes the achievable *actual* in production. Practitioners should invest in observability (at least to the metrics+logs+traces level) and in continual learning *together*, and should prefer bounded-latency online adaptation to bursty periodic retraining wherever a label signal and a latency budget coexist.

Future work follows four directions. First, *label efficiency*: extend the online comparison to semi- and self-supervised drift adaptation for settings where ground-truth labels are scarce or delayed, removing the present dependence on a prompt label signal. Second, *topological generalisation*: extend the testbed beyond the present nine-service mesh to larger and production-scale call graphs, testing whether trace-driven localisation remains complete—as it is here—as the mesh deepens further and dependency structures grow more irregular. Third, *drift diversity*: broaden the regime set beyond latency/scale/load to include schema changes, dependency re-wiring and seasonal patterns, and pair the empirical study with explicit drift-detection signals. Fourth, *operational integration*: close the loop from online detection through trace-driven localisation to automated remediation, drawing on the LLM- and Bayesian-network RCA directions of Pedrosa et al. [37] and the holistic fusion of Han et al. [11], so that a continually-current detector feeds a continually-current responder. A fifth direction concerns *explicit drift signalling*: the present online model adapts implicitly through its normaliser and champion bandit, but pairing it with a dedicated drift detector would let the system distinguish virtual from real drift at runtime—precisely the distinction that §6.4 shows determines whether refitting or re-normalising is the right response—and could trigger heavier adaptation only when a genuine concept change is detected, conserving the steady-state CPU that is the online model’s sole cost disadvantage. A sixth direction is to *evaluate the streaming and LLM additions* introduced here (§4.3): quantify the served Qwen2.5-3B detector, base versus LoRA-tuned [24], against the linear online model on the same prequential stream; measure the Kafka backbone’s tail latency, consumer lag and loss-on-rebalance under load relative to the deterministic in-process harness; and harden the LLM detector against the telemetry-borne prompt-injection exposure noted in §7 [39]. Finally, the comparison should be extended to *multi-seed and multi-trace-load* runs to convert the large but single-seed effect sizes reported here into formal significance intervals, and to confirm that the paradigm ordering is invariant to the specific synthetic stream.

Taken together, these directions share a single thesis, which is the contribution we most wish to carry forward: in a non-stationary world, a detector is never finished being trained. The observability community has invested heavily in *seeing* more of the system; the evidence here is that comparable investment in *relearning* what is seen is what converts that visibility into reliable detection.

Acknowledgements. The authors thank colleagues at the School of Built Environment, Engineering and Computing, Leeds Beckett University, for discussion and

feedback. All experiments used open-source tooling; the result datasets underpinning every table are available for reproducibility.

Declarations

- **Funding.** No specific grant funded this work.
- **Competing interests.** The authors declare no competing interests.
- **Ethics approval.** The research involves no human participants or personal data; all data are generated on a self-contained testbed with fault injection confined to an isolated environment, in line with Leeds Beckett University research-ethics guidance.
- **Data and code availability.** The testbed configuration and the result datasets supporting the findings are available from the authors on reasonable request.

Appendix A Full feature listing

Table A1 enumerates the engineered features by configuration. All features are window-local aggregates computed over the fixed-length detection window; configurations are nested, so the features of a configuration are the union of its own row and all rows above it. The dimensionalities (10, 14, 18, 23) correspond to the cumulative counts of Table 1.

Table A1 Engineered features by pillar. Counts are per-pillar additions; cumulative totals are 10 (C1), 14 (C2), 18 (C3), 23 (C4).

Pillar (config)	#	Features
Metrics (C1)	10	CPU utilisation; memory utilisation; request rate; error rate; p50 latency; p95 latency; p99 latency; network bytes in; network bytes out; pod restart count
Logs (C2)	4	total log volume; error-level log count; warning-level log count; log-template diversity (distinct templates per window)
Traces (C3)	4	span rate; mean span duration; error-span rate; originating-error-span indicator (root of the error tree)
Events + history (C4)	5	OOMKilled event count; CrashLoopBackOff event count; lifecycle restart-event count; rolling-baseline deviation (short horizon); rolling-baseline deviation (long horizon)

Appendix B Experimental constants

Table B2 records the fixed parameters of the prequential RQ3 experiment, held constant across all telemetry configurations and learning policies.

Table B2 Fixed experimental parameters.

Parameter	Value
Episodes	320
Windows evaluated	11,520 (2,880 warm-up R_0 + 8,640 future)
Services	movie-service, actor-service, review-service
Regime sequence	$R_0 \rightarrow R_1 \rightarrow R_2 \rightarrow R_3$
Periodic retrain interval τ	every 500 windows (17 refits)
Periodic training buffer B	2,880 windows
Online candidate pool $\mathcal{H}$	$\eta_0 \in \{0.01, 0.05, 0.1\}$, $\alpha \in \{10^{-4}, 10^{-3}\}$
Evaluation protocol	prequential (test-then-train)
Random seed	fixed (single deterministic run)

References

- [1] Cloud Native Computing Foundation: Cloud native 2024: Approaching a decade of code, cloud, and change. Technical report, Cloud Native Computing Foundation (2024). <https://www.cncf.io/reports/cncf-annual-survey-2024/>
- [2] DORA: Accelerate state of DevOps report 2024. Technical report, Google Cloud (2024). <https://dora.dev/research/2024/>
- [3] Mohammad, M.: Resilient microservices: A systematic review of recovery patterns, strategies, and evaluation frameworks. arXiv preprint arXiv:2512.16959 (2025) <https://doi.org/10.48550/arXiv.2512.16959>
- [4] Islam, M.S., Pourmajidi, W., Zhang, L., Steinbacher, J., Erwin, T., Miranskyy, A.: Anomaly detection in a large-scale cloud platform. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 150–159. IEEE, ??? (2021). <https://doi.org/10.1109/ICSE-SEIP52600.2021.00024>
- [5] Mart, O., Negru, C., Pop, F., Castiglione, A.: Observability in kubernetes cluster: Automatic anomalies detection using prometheus. In: 2020 IEEE 22nd International Conference on High Performance Computing and Communications (HPCC/SmartCity/DSS), pp. 565–570. IEEE, ??? (2020). <https://doi.org/10.1109/HPCC-SmartCity-DSS50907.2020.00071>
- [6] Kosińska, J., Baliś, B., Konieczny, M., Malawski, M., Zieliński, S.: Toward the observability of cloud-native applications: The overview of the state-of-the-art. IEEE Access **11**, 73036–73058 (2023) <https://doi.org/10.1109/ACCESS.2023.3281860>
- [7] Boten, A.: Cloud-Native Observability with OpenTelemetry. Packt Publishing, Birmingham, UK (2022)
- [8] Chapman, R., Holmes, P.: Observability with Grafana: Monitor, Control, and

Visualize Your Kubernetes and Cloud Platforms. Packt Publishing, Birmingham, UK (2023)

- [9] Lomio, F., Moreschini, S., Li, X., Lenarduzzi, V.: Anomaly detection in cloud-native systems. In: 2022 48th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 100–103. IEEE, ??? (2022). <https://doi.org/10.1109/SEAA56994.2022.00022>
- [10] Dkmak, G., Can, B., Sevinc, O., Egeli, C.B., Baday, F., Cetintav, B.: Ai-driven anomaly detection in cloud-native microservices: The night’s watch algorithm. *Applied Sciences* **15**(23), 12762 (2025) <https://doi.org/10.3390/app152312762>
- [11] Han, Y., Du, Q., Huang, Y., Li, P., Shi, X., Wu, J., Fang, P., Tian, F., He, C.: Holistic root cause analysis for failures in cloud-native systems through observability data. *IEEE Transactions on Services Computing* **17**(6), 3789–3802 (2024) <https://doi.org/10.1109/TSC.2024.3476599>
- [12] Wang, P., Xu, J., Ma, M., Lin, W., Pan, D., Wang, Y., Chen, P.: CloudRanger: Root cause identification for cloud native systems. In: 2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid), pp. 492–502. IEEE, ??? (2018). <https://doi.org/10.1109/CCGRID.2018.00076>
- [13] Chen, X., Kou, J., Li, H., Zhang, Y., Ma, J., Li, C., Tu, B.: End-to-end anomaly detection of service function chain through multi-source data in cloud-native systems. *Computers & Security* **155**, 104461 (2025) <https://doi.org/10.1016/j.cose.2025.104461>
- [14] Raeiszadeh, M., Estrada-Solano, F., Glitho, R.H.: Anomalies detection in cloud-native architectures: Background, state-of-the-art, and research directions. *IEEE Communications Magazine* (2026). Early Access
- [15] Landauer, M., Onder, S., Skopik, F., Wurzenberger, M.: Deep learning for anomaly detection in log data: A survey. *arXiv preprint arXiv:2207.03820* (2022) <https://doi.org/10.48550/arXiv.2207.03820>
- [16] Sarfraz, M.S., Chen, M.-Y., Layer, L., Peng, K., Koulakis, M.: Position: Quo vadis, unsupervised time series anomaly detection? In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. PMLR, vol. 235, pp. 43461–43476 (2024)
- [17] Honeycomb: Building a leading observability practice. Technical report, Honeycomb.io (2022)
- [18] Hausenblas, M.: Return on investment driven observability. *arXiv preprint arXiv:2303.13402* (2023) <https://doi.org/10.48550/arXiv.2303.13402>
- [19] Huang, H., Chen, C., Chen, K., Chen, P., Yu, G., He, Z., Wang, Y., Zhang,

- H., Zhou, Q.: Mint: Cost-efficient tracing with all requests collection via commonality and variability analysis. In: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 683–697 (2025). <https://doi.org/10.1145/3669940.3707287>
- [20] Hrusto, A., Ali, N.B., Engström, E., Wang, Y.: Monitoring data for anomaly detection in cloud-based systems: A systematic mapping study. *ACM Transactions on Software Engineering and Methodology* **35**(4), 91 (2026) <https://doi.org/10.1145/3744556>
- [21] Kreps, J., Narkhede, N., Rao, J.: Kafka: A distributed messaging system for log processing. In: Proceedings of the NetDB Workshop (6th International Workshop on Networking Meets Databases), pp. 1–7 (2011)
- [22] Qwen Team: Qwen2.5 technical report. arXiv preprint arXiv:2412.15115 (2025)
- [23] Ollama: Ollama: Get up and running with large language models locally. Software, ollama.com (2024)
- [24] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: Low-rank adaptation of large language models. In: International Conference on Learning Representations (ICLR) (2022)
- [25] Arora, S., Rani, R., Saxena, N.: A systematic review on detection and adaptation of concept drift in streaming data using machine learning techniques. *WIREs Data Mining and Knowledge Discovery* **14**(4) (2024) <https://doi.org/10.1002/widm.1536>
- [26] Lukats, D., Zielinski, O., Hahn, A., Stahl, F.T.: A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams. *International Journal of Data Science and Analytics* **19** (2025) <https://doi.org/10.1007/s41060-024-00620-y>
- [27] Wang, F., Jiang, Y., Zhang, R., Wei, A., Xie, J., Pang, X.: A survey of deep anomaly detection in multivariate time series: Taxonomy, applications, and directions. *Sensors* **25**(1), 190 (2025) <https://doi.org/10.3390/s25010190>
- [28] Chen, J., Liu, F., Jiang, J., Zhong, G., Xu, D., Tan, Z., Shi, S.: TraceGra: A trace-based anomaly detection for microservice using graph deep learning. *Computer Communications* **204**, 109–117 (2023) <https://doi.org/10.1016/j.comcom.2023.03.028>
- [29] Akmeemana, L., Attanayake, C., Faiz, H., Wickramanayake, S.: GAL-MAD: Towards explainable anomaly detection in microservice applications using graph attention networks. arXiv preprint arXiv:2504.00058 (2025) <https://doi.org/10.48550/arXiv.2504.00058>

- [30] Guan, W., Cao, J., Qian, S., Gao, J.: LogLLM: Log-based anomaly detection using large language models. arXiv preprint arXiv:2411.08561 (2024) <https://doi.org/10.48550/arXiv.2411.08561>
- [31] Zhuang, J., Yan, L., Zhang, Z., Wang, R., Zhang, J., Gu, Y.: See it, think it, sorted: Large multimodal models are few-shot time series anomaly analyzers. arXiv preprint arXiv:2411.02465 (2024) <https://doi.org/10.48550/arXiv.2411.02465>
- [32] Wang, P., Zhang, X., Chen, Y., Cao, Z.: Unsupervised microservice system anomaly detection via contrastive multi-modal representation clustering. *Information Processing & Management* **62** (2025) <https://doi.org/10.1016/j.ipm.2024.104013>
- [33] Wette, S., Heinrichs, F.: OML-AD: Online machine learning for anomaly detection in time series data. arXiv preprint arXiv:2409.09742 (2024) <https://doi.org/10.48550/arXiv.2409.09742>
- [34] Li, J., Malialis, K., Polycarpou, M.M.: Autoencoder-based anomaly detection in streaming data with incremental learning and concept drift adaptation. In: *International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8 (2023). <https://doi.org/10.1109/IJCNN54540.2023.10191328>
- [35] Bakhtin, A., Nyyssölä, J., Wang, Y., Ahmad, N., Ping, K., Esposito, M., Mäntylä, M., Taibi, D.: LO2: Microservice API anomaly dataset of logs and metrics. In: *Proceedings of the 21st International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)* (2025). <https://doi.org/10.1145/3727582.3728682>
- [36] Pham, L., Zhang, H., Ha, H., Salim, F., Zhang, X.: RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data. In: *Companion Proceedings of the ACM Web Conference (WWW)* (2025). <https://doi.org/10.1145/3701716.3715290>
- [37] Pedroso, D.F., Almeida, L., Pulcinelli, L.E.G., Aisawa, W.A.A., Dutra, I., Bruschi, S.M.: Anomaly detection and root cause analysis in cloud-native environments using large language models and bayesian networks. *IEEE Access* **13**, 77550–77571 (2025) <https://doi.org/10.1109/ACCESS.2025.3556193>
- [38] Amazon Web Services: Cloud-native observability (eBook 5). Technical report, AWS Marketplace (2022)
- [39] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pp. 79–90 (2023)

- [40] Faseeha, U., Syed, H.J., Samad, F., Zehra, S., Ahmed, H.: Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms. *IEEE Access* **13** (2025) <https://doi.org/10.1109/ACCESS.2025.3562125>
- [41] Honeycomb: A beginner’s guide to observability. Technical report, Honeycomb.io (2021)
- [42] OpenTelemetry Authors: OpenTelemetry Specification. <https://opentelemetry.io/docs/specs/otel/>. Cloud Native Computing Foundation; specification v1.x (2024)
- [43] Wu, Y., Yu, G., Jiang, Z., Li, Y., Lyu, M.R.: Trace sampling 2.0: Code knowledge enhanced span-level sampling for distributed tracing. *arXiv preprint arXiv:2509.13852* (2025) <https://doi.org/10.48550/arXiv.2509.13852>
- [44] Ashok, S., Harsh, V., Godfrey, B., Mittal, R., Parthasarathy, S., Shwartz, L.: TraceWeaver: Distributed request tracing for microservices without application modification. In: *Proceedings of the ACM SIGCOMM 2024 Conference* (2024). <https://doi.org/10.1145/3651890.3672254>
- [45] Xia, F., Chen, X., Yu, S., Hou, M., Liu, M., You, L.: Coupled attention networks for multivariate time series anomaly detection. *IEEE Transactions on Emerging Topics in Computing* (2024) <https://doi.org/10.1109/TETC.2023.3280577>
- [46] Choi, K., Yi, J., Mok, J., Yoon, S.: Self-supervised time-series anomaly detection using learnable data augmentation. *arXiv preprint arXiv:2406.12260* (2024) <https://doi.org/10.48550/arXiv.2406.12260>
- [47] Singal, A., Pathak, D., Ray, K., George, F., Verma, M., Moogi, P.: Metric criticality identification for cloud microservices. *arXiv preprint arXiv:2501.03547* (2025) <https://doi.org/10.48550/arXiv.2501.03547>
- [48] Wagner, D., Nair, A., Franks, B.J., *et al.*: Formally exploring time-series anomaly detection evaluation metrics. *arXiv preprint arXiv:2510.17562* (2025) <https://doi.org/10.48550/arXiv.2510.17562>
- [49] Pospieszny, P., Mormul, W., Szyndler, K., Kumar, S.: ADALog: Adaptive unsupervised anomaly detection in logs with self-attention masked language model. In: *10th International Conference on Machine Learning Technologies (ICMLT)*, pp. 248–256 (2025). <https://doi.org/10.1109/ICMLT65785.2025.11193311>
- [50] Zhang, Q., Lyu, N., Liu, L., Wang, Y., Cheng, Z., Hua, C.: Graph neural AI with temporal dynamics for comprehensive anomaly detection in microservices. *arXiv preprint arXiv:2511.03285* (2025) <https://doi.org/10.48550/arXiv.2511.03285>

- [51] Al-Omari, T.: Context-aware anomaly detection in microservices using GCN-encoded trace graphs and LSTM-AE metrics with local and global embeddings. *Engineering, Technology & Applied Science Research* **15**(6), 29277–29283 (2025) <https://doi.org/10.48084/etasr.13590>
- [52] Zhang, W., Xin, R., Chen, P., Li, X., Bian, A., Zhao, Y., Zhao, Z.: MODIFY: A multi-modal anomaly diagnosis framework with diffusion-enhanced adaptive fusion in microservices. *Journal of Systems and Software* **237**, 112844 (2026) <https://doi.org/10.1016/j.jss.2026.112844>
- [53] Zhang, L., Jia, T., Jia, M., Wu, Y., Liu, A., Yang, Y., Wu, Z., Hu, X., Yu, P.S., Li, Y.: A survey of AIOps for failure management in the era of large language models. *arXiv preprint arXiv:2406.11213* (2024) <https://doi.org/10.48550/arXiv.2406.11213>
- [54] Xu, Y., Ge, J., Tang, H., Ding, S., Li, T., Li, H.: System states forecasting of microservices with dynamic spatio-temporal data. *arXiv preprint arXiv:2408.07894* (2024) <https://doi.org/10.48550/arXiv.2408.07894>
- [55] Wang, T., Qi, G.: A comprehensive survey on root cause analysis in (micro)services: Methodologies, challenges, and trends. *arXiv preprint arXiv:2408.00803* (2024) <https://doi.org/10.48550/arXiv.2408.00803>
- [56] Pham, L., Ha, H., Zhang, H.: Root cause analysis for microservice system based on causal inference: How far are we? In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 706–715 (2024). <https://doi.org/10.1145/3691620.3695065>
- [57] Yao, Z., Pei, C., Chen, W., Wang, H., Su, L., Jiang, H., Xie, Z., Nie, X., Pei, D.: Chain-of-event: Interpretable root cause analysis for microservices through automatically learning weighted event causal graph. In: *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*, pp. 50–61 (2024). <https://doi.org/10.1145/3663529.3663827>
- [58] Pham, L., Ha, H., Zhang, H.: BARO: Robust root cause analysis for microservices via multivariate bayesian online change point detection. *Proceedings of the ACM on Software Engineering* **1**(FSE), 2214–2237 (2024) <https://doi.org/10.1145/3660805>
- [59] Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., Zeng, J., Ghosh, S., Zhang, X., Zhang, C., Lin, Q., Rajmohan, S., Zhang, D., Xu, T.: Automatic root cause analysis via large language models for cloud incidents. In: *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)* (2024). <https://doi.org/10.1145/3627703.3629553>
- [60] Zhang, L., Zhai, Y., Jia, T., Duan, C., Yu, S., Gao, J., Ding, B., Wu, Z., Li,

- Y.: ThinkFL: Self-refining failure localization for microservice systems via reinforcement fine-tuning. arXiv preprint arXiv:2504.18776 (2025) <https://doi.org/10.48550/arXiv.2504.18776>
- [61] Hovakimyan, G., Bravo, J.M.: Evolving strategies in machine learning: A systematic review of concept drift detection. *Information* **15**(12), 786 (2024) <https://doi.org/10.3390/info15120786>
 - [62] Li, J., Malialis, K., Panayiotou, C.G., Polycarpou, M.M.: Unsupervised incremental learning with dual concept drift detection for identifying anomalous sequences. In: *International Joint Conference on Neural Networks (IJCNN)* (2024). <https://doi.org/10.1109/IJCNN60899.2024.10649991>
 - [63] Leest, J., Gerostathopoulos, I., Lago, P., Raibulet, C.: Monitoring and observability of machine learning systems: Current practices and gaps. arXiv preprint arXiv:2510.24142 (2025) <https://doi.org/10.48550/arXiv.2510.24142>
 - [64] Islam, M.S., Rakha, M.S., Pourmajidi, W., Sivaloganathan, J., Steinbacher, J., Miransky, A.V.: Anomaly detection in large-scale cloud systems: An industry case and dataset. In: *47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 377–388 (2025). <https://doi.org/10.1109/ICSE-SEIP66354.2025.00039>
 - [65] Ping, K., Mazhar, H.B., Wang, Y., Song, Y., Mäntylä, M.V.: AnoMod: A dataset for anomaly detection and root cause analysis in microservice systems. arXiv preprint arXiv:2601.22881 (2026) <https://doi.org/10.48550/arXiv.2601.22881>
 - [66] Owotogbe, J., Kumara, I., Heuvel, W.-J., Tamburri, D.A.: Chaos engineering: A multi-vocal literature review. *ACM Computing Surveys* **58**(7), 164–116444 (2026) <https://doi.org/10.1145/3777375>